

Julius-Maximilians-Universität Würzburg

Philosophische Fakultät

Digital Humanities

Freies digitale Projekt für Digital Humanities

**Aufbau, Erschließung und prototypische Suchbereitstellung eines
Korpus russischsprachiger Diskmags auf Grundlage von ZXpress**

vorgelegt von Artem Stoianov

Studiengang: Digital Humanities

Matrikelnummer: 2600511

Betreuer: Dr. phil. Torsten Roeder

Abgabedatum: 08.06.2026

Inhaltsverzeichnis

1. Einleitung und Forschungsrahmen	3
2. Projektgenese und konzeptuelle Ausgangslage	4
3. Quellenlage, Kontakt zu ZXpress und Entscheidung für eigenes Scraping	6
4. Entwicklung der Scraping-Pipeline	9
5. Korpusmodell und Datenorganisation	12
6. Validierung, Bereinigung und Sonderfälle	17
7. Endvalidierung und Abschluss der Korpusstruktur	23
8. Lucene-Indexierung und Entwicklung des Suchprototyps	27
9. Erster Versuch eines eigenen FCS-/SRU-Endpoints (scripts/FCS/)	36
10. Zweiter Versuch eines FCS-Endpoints (scripts/FCS_Server/)	51
Quellen- und Ressourcenverzeichnis	60

1. Einleitung und Forschungsrahmen

Ziel des Projekts ist der Aufbau eines digital erschlossenen, durchsuchbaren Korpus russischsprachiger ZX-Diskmags auf Grundlage der Plattform ZXpress.

Im Zentrum stand damit ein Bestand, der für die Digital Humanities in mehrfacher Hinsicht relevant ist: Er gehört zur born-digital Kulturgeschichte, ist eng mit der osteuropäischen ZX-Spectrum-Szene verbunden und liegt zugleich in einer Form vor, die langfristig gefährdet ist. Viele Texte sind zwar noch auf der Website erreichbar, doch fehlt ihnen eine stabile, zitierfähige und wissenschaftlich nutzbare Infrastruktur. Das Projekt reagiert damit auf ein typisches Problem digitaler Kulturüberlieferung: Inhalte sind vorhanden, aber weder nachhaltig archiviert noch standardisiert erschlossen.

Aus fachlicher Sicht verfolgte das Projekt drei Hauptziele. Erstens sollte ein belastbarer Korpus aus Magazinen, Ausgaben und Artikeln aufgebaut werden. Zweitens sollte dieser Korpus über eine eigene Volltextsuche mit Kontextanzeige nutzbar werden. Drittens sollte perspektivisch eine Pseudo-Anbindung an Federated Content Search (FCS) erfolgen, um die Ressource in einen standardisierten wissenschaftlichen Suchzusammenhang zu überführen.

Inhaltlich ist der Bestand besonders interessant, weil die Diskmags nicht nur technische Informationen enthalten, sondern auch Humor, Szenesprache, regionale Netzwerke, literarische Formen, Anzeigen, Debatten und Selbstbeschreibungen einer spezifischen Computerkultur. Die Texte sind damit nicht bloß Träger von Informationen, sondern auch Ausdruck einer sozialen und medialen Praxis. Genau diese Verbindung aus technischen und kulturellen Aspekten machte den Bestand für das Projekt besonders interessant.

Von Beginn an war außerdem klar, dass das Vorhaben nicht nur als technische Übung verstanden werden durfte. Der Korpus sollte nicht einfach primitiv durchsuchbar werden, sondern so aufgebaut sein, dass Herkunft, Struktur, Metadaten und Verarbeitungsschritte nachvollziehbar bleiben. Die Frage war daher nicht nur, wie man Texte extrahiert und indiziert, sondern auch, wie aus einer flüchtigen Webquelle ein wissenschaftlich brauchbares Arbeitsobjekt werden kann.

Aus dieser Grundfrage ergaben sich die ersten konzeptuellen Überlegungen: Welche Art von Korpus sollte überhaupt entstehen, welche Suchformen wären dafür sinnvoll und in welcher technischen Form könnte die Ressource später bereitgestellt werden (*vgl. Kap. 2*)?

2. Projektgenese

Die erste Projektphase war von vielen konzeptuellen Überlegungen geprägt.

Ausgangspunkt war zunächst die Idee, eine Volltextsuche für den Bestand aufzubauen und diese später in Richtung strukturierterer Suchformen zu erweitern.

In den frühen Besprechungen wurde deshalb nicht sofort mit FCS im engeren Sinn begonnen, sondern mit grundsätzlichen Fragen:

1. Welche Texte liegen vor?
2. Welche Suchformen sind für sie sinnvoll?
3. Wie kann ein Server überhaupt mitteilen, welche Dokumente er besitzt und welche Arten von Antworten er auf Suchanfragen zurückgeben kann?

Bereits hier tauchten die Begriffe KWIC, SRU, CQL und FCS auf, allerdings noch als Zielhorizont.

In dieser Konzeptphase zeigte sich auch die fachliche Logik des Projekts. Der Bestand sollte nicht nur technisch abgefragt werden, sondern als Korpus modelliert werden. Dazu gehörte die Überlegung, Inhalte in eine strukturierte Form zu überführen, Metadaten konsistent zu erfassen und langfristig auch TEI als mögliches Archiv- und Austauschformat mitzudenken. Zugleich war klar, dass ein vollständiger TEI-Workflow für den ersten Prototyp zu aufwendig gewesen wäre. Deshalb wurde ein pragmatischer Aufbau gewählt: zunächst Korpusgewinnung, dann Suchfunktion, anschließend Standardisierung und Endpoint-Anbindung.

Eine wichtige Rolle spielte die Frage nach der Ergebnisdarstellung. Für das Projekt war nicht irgendeine Trefferliste ausreichend. Gesucht wurde ein Format, in dem ein Treffer nicht isoliert erscheint, sondern zusammen mit seinem Kontext sichtbar wird. Die Entscheidung für eine KWIC-orientierte Ausgabe war deshalb inhaltlich motiviert: Für die spätere FCS-Kompatibilität und für die praktische Arbeit mit dem Korpus war es sinnvoll, das gefundene Wort zusammen mit linkem und rechtem Kontext auszugeben. Dadurch wurde die Suchmaschine von Anfang an nicht nur als Zählwerkzeug, sondern als Lese- und Analysehilfe gedacht (*vgl. Kap. 8.5–8.7*).

In dieser Konzeptphase fiel die Entscheidung, ZXpress als zentrale Quelle zu nutzen. Die Website stellte allerdings keinen fertigen Forschungsdatensatz bereit, sondern nur das Material, aus dem ein solcher Datensatz erst erzeugt werden musste.. Genau aus dieser Spannung zwischen vorhandener Quelle und fehlender Infrastruktur entwickelte sich die spätere Pipeline des Projekts (*vgl. Kap. 3-4*).

Der nächste praktische Schritt bestand deshalb darin, die reale Quellenlage zu klären. Dabei stellte sich vor allem die Frage, ob die Daten direkt vom Betreiber bezogen werden konnten oder ob der Bestand technisch selbst erschlossen werden musste.

3. Quellenlage, Kontakt zu ZXpress und Entscheidung für eigenes Scraping

Die Arbeit am Korpus begann mit der Frage, ob ein direkter und möglichst schonender Zugang zu den Quelldaten möglich wäre.

Da die Plattform ZXpress (zxpress.ru/ezines.php) bereits eine große Sammlung russischsprachiger Diskmags bereitstellt, lag es zunächst nahe, nicht sofort eine eigene technische Erschließung zu entwickeln, sondern zuerst zu prüfen, ob die Daten in strukturierter Form bezogen werden könnten. Dies war sowohl aus praktischen als auch aus wissenschaftlichen Gründen sinnvoll: Ein direkter Export der Datenbank oder ein geordnetes Archiv der Texte hätte nicht nur Arbeitsschritte vereinfacht, sondern auch die Provenienz der Materialien transparenter gemacht.

Aus diesem Grund wurde zunächst Kontakt mit dem Betreiber von ZXpress aufgenommen. In dieser Kommunikation wurde das Projekt als akademisches, nicht-kommerzielles Vorhaben vorgestellt, das auf die Bewahrung und wissenschaftliche Erschließung digitaler Kulturquellen zielt. Konkret wurde erklärt, dass die Texte in einen wissenschaftlich nutzbaren Korpus überführt und perspektivisch über Federated Content Search (FCS) zugänglich gemacht werden sollten. Dabei wurde ausdrücklich betont, dass die Rechte an den Materialien bei den ursprünglichen Rechteinhabern verbleiben und dass Herkunft und Quellenangaben sichtbar erhalten bleiben sollten.

Die Rückmeldung des Betreibers war grundsätzlich offen und kooperativ. Dabei wurde jedoch zugleich deutlich, dass kein unmittelbar fertiger Forschungsdatensatz vorlag. Der Betreiber wies darauf hin, dass die Texte im Grunde bereits über die Website zugänglich seien, insbesondere über druckfreundliche Ansichten, und dass ein automatisierter Abruf über Artikel-IDs prinzipiell möglich sei. Gleichzeitig wurde ein zentrales Problem angesprochen, das für das gesamte Projekt wichtig blieb: Ein Teil der Texte enthält Erkennungs- bzw. Übertragungsfehler, insbesondere Verwechslungen zwischen kyrillischen und lateinischen Buchstaben mit ähnlicher Form (zufällig und auch aus Spaß). Schon an dieser Stelle zeigte sich also, dass die Quelle texttechnisch nicht homogen war.

In einer weiteren Nachricht wurde von Projektseite vorgeschlagen, die Materialien im Idealfall in zwei Formen zu erhalten: zum einen als vollständigen SQL-Dump mit Metadaten und relationaler Struktur, zum anderen als parallelen TXT-Export der Artikeltexte. Diese Anfrage war inhaltlich bereits stark auf die spätere Verarbeitung ausgerichtet. Sie bezog sich

auf Metadaten wie Magazinname, Ausgabennummer, Artikel-ID, Titel, Datum, Publikationsform und Ort. Zusätzlich wurde die Frage der Quellen- und Rechtekennzeichnung angesprochen, um eine spätere wissenschaftliche Nutzung mit sichtbarer Herkunftsmarkierung vorzubereiten.

Ein solcher strukturierter Export kam jedoch nicht zustande. Der Kontakt brach an dieser Stelle faktisch ab, oder blieb zumindest ohne weitere verwertbare Datenlieferung. Damit veränderte sich die Projektsituation grundlegend: Aus einer möglichen Übernahme bereits strukturierter Daten wurde die Notwendigkeit, den Bestand selbst aus der Webpräsentation zu rekonstruieren.

Diese Entscheidung war nicht bloß eine technische Ausweidlösung, sondern prägte die weitere Durchführung des gesamten Projekts. Es trat nun ein eigenständiger Prozess der Datenerhebung, Strukturierung und Validierung.

Die Entscheidung für eigenes Scraping war dabei aus mehreren Gründen sinnvoll. Erstens existierte keine offizielle API oder dokumentierte Export-Schnittstelle.

Zweitens zeigte die Website eine ausreichend regelmäßige Struktur, um automatisiert bearbeitet zu werden. Die Grundhierarchie Magazin → Ausgabe → Artikel ließ sich sowohl in der Navigation als auch in den URL-Mustern und HTML-Strukturen erkennen.

Drittens boten die druckfreundlichen Ansichten der Artikel einen vergleichsweise sauberen Zugriff auf die eigentlichen Texte, da sie weniger Navigations- und Layoutballast enthielten als die normalen Seitenansichten. Die Website fungierte damit faktisch als eine Art „Quasi-API“.

Damit war allerdings auch klar, dass der Scraping-Prozess selbst Teil der wissenschaftlichen Arbeit werden würde. Die Daten mussten heruntergeladen und in eine reproduzierbare Struktur überführt werden. Das betraf sowohl die technische Ebene, also URL-Muster, HTML-Parsen, Dateiorganisation und Encoding und auch die editorische Ebene, etwa den Umgang mit unvollständigen Datierungen, problematischen Zeichenfolgen oder nicht standardisierten Metadaten.

Aus dieser Lage ergab sich die spätere Entscheidung für eine mehrstufige Pipeline mit JSON-Metadaten, Rohdaten, Validierungsskripten und Health-Checks (*vgl. Kap. 4-7*).

Rückblickend war diese Phase ein methodisch wichtiger Übergang. Aus einer zunächst eher konzeptuellen Such- und Infrastrukturidee wurde nun eine konkrete Arbeit an den Daten selbst. Gerade weil kein fertiger Datensatz bereitstand, musste die Quelle selbst zum Gegenstand technischer Entscheidungen werden. Die spätere Suchmaschine und die spätere FCS-Anbindung basierten somit auf einem selbst erzeugten, schrittweise geprüften und dokumentierten Korpus (*vgl. Kap. 8-10*).

Damit verschob sich der Schwerpunkt des Projekts zum ersten Mal von der konzeptuellen Planung zur praktischen Korpusgewinnung. Der nächste Arbeitsschritt war daher die Entwicklung einer Pipeline, mit der sich die Inhalte von ZXpress systematisch in einen lokalen Bestand überführen ließen.

4. Entwicklung der Scraping-Pipeline

Nach der Entscheidung für eigenes Scraping musste ein Verfahren entwickelt werden, mit dem die Inhalte der Website systematisch in einen lokalen Korpus überführt werden konnten.

Die Entwicklung verlief dabei nicht in einem einzigen stabilen Schritt, sondern in mehreren Phasen.

Im Repository lassen sich zwei Scraping-Generationen unterscheiden: ein früherer Strang im Root-Ordner `archive/zxpress/` und eine spätere, stärker modularisierte Pipeline unter `scripts/light/`.

Die erste Generation liegt in `archive/zxpress/`.

Dort finden sich mit `scrape_catalog.py`, `scrape_magazine.py`, `scrape_articles.py`, `build_indexes.py`, `fill_missing.py`, `models.py` und `utils.py` bereits alle wesentlichen Bestandteile eines frühen Scraping- und Korpusaufbaus. Dieser Strang war funktional und legte die Grundstruktur fest, mit der Magazine, Ausgaben und Artikel aus der Website gewonnen wurden. Die Seite wurde gezielt über ihre stabile Hierarchie angesprochen. Magazinseiten lieferten die Ausgabenlisten, Artikelseiten wurden über IDs adressiert, und druckfreundliche Varianten wurden als bevorzugte Textquelle genutzt.

Im weiteren Projektverlauf erwies sich dieser frühe Ansatz jedoch als zu kompakt. Mit wachsendem Umfang des Korpus traten Probleme auf, die nicht mehr in einem einzigen Scraping-Schritt sinnvoll bearbeitet werden konnten: fehlende Datierungen, leere Magazine, unvollständige Listings, Dubletten, Sonderzeichen in Namen und fehlende Artikeldaten. Damit wurde deutlich, dass der Korpusaufbau Datenerhebung sowie Nachbearbeitung, Kontrolle und Reparatur einschließen musste.

Aus dieser Situation entstand die spätere Pipeline unter `scripts/light/`.

Dieser Bereich ist deutlich modularer aufgebaut und entspricht eher einem reproduzierbaren Workflow als einem einzelnen Scraper. Zentral ist dabei `run_light_pipeline.py`, das die einzelnen Schritte bündelt: zunächst das Erzeugen der Ausgabenlisten, dann das Patchen der Metadaten, anschließend das Nachladen der Artikel und schließlich die Validierung des erzeugten Bestands. Im Unterschied zum frühen `zxpress`-Strang wurde der Korpusaufbau damit als mehrphasiger Prozess organisiert.

Der erste Schritt dieser späteren Pipeline liegt in `scrape_issue_listing_light.py`.

Dieses Skript liest eine Magazinseite ein, extrahiert die vorhandenen Ausgaben und erzeugt gleichzeitig strukturierte Magazinmetadaten. Technisch notwendig ist dabei, dass bevorzugt mit der linken Inhaltsspalte (`col-left`) gearbeitet wird, um Navigations- und Layoutbestandteile auszublenden. Auf diese Weise entstehen `magazine.json` auf Magazinebene und `listing.json` auf Ausgabebene.

Bereits in diesem Schritt werden die Artikellinks vorbereitet. Ein zentraler Ausschnitt ist:

```
if "article.php?id=" in href and current_label:
    url = abs_url(href)
    aid = int(href.split("=")[-1])
    issues[current_label]["articles"].append({
        "article_id": aid,
        "article_url": url,
        "print_url": abs_url(f"print.php?id={aid}"),
        "title_link": title
    })
```

Die Pipeline speichert also nicht nur die Artikellinks, sondern legt gleichzeitig auch die `print_url` an, die später für den eigentlichen Textabruf genutzt wird. Dadurch wird die Logik der Website schon früh in eine eigene Zwischenstruktur übertragen.

Der eigentliche Textabruf erfolgt in `fetch_articles_light.py`.

Dieses Skript arbeitet mit den zuvor gespeicherten Artikel-IDs und lädt die Texte gezielt über die druckfreundlichen Ansichten. Der relevante Mechanismus ist sehr direkt:

```
def fetch_print_text(sess, article_id):
    url = f"{BASE_URL}/print.php?id={article_id}"
    r = sess.get(url)
    r.encoding = "utf-8"
    soup = BeautifulSoup(r.text, "html.parser")
    pre = soup.find("pre", id="text")
```

Die Print-Ansichten wurden im Projekt deshalb bevorzugt, weil sie deutlich weniger störendes Layout enthielten als die normalen Artikelseiten. Für das Parsing war das robuster und einfacher zu kontrollieren. Aus heutiger Sicht bestätigt dieser Zugriff die frühere Beobachtung, dass die Website technisch wie eine Art „Quasi-API“ behandelt werden konnte.

Neben dem Text speichert `fetch_articles_light.py` auch die Metadaten des Artikels.

Pro Artikel entstehen `text.txt` und `meta.json`. Dabei werden Kontextinformationen aus Magazin- und Ausgabenebene übernommen und um artikelspezifische Felder wie Reihenfolge, Titel, `print_url`, `article_url` und Abrufzeitpunkt ergänzt. Auf diese Weise wurde jeder Artikel als eigenständige Einheit des Korpus gespeichert.

Rückblickend lässt sich die Entwicklung der Scraping-Pipeline daher als Übergang von einer frühen funktionsfähigen Erschließung zu einem kontrollierten Workflow beschreiben.

Nach der Stabilisierung dieses Workflows stellte sich zugleich die Frage, in welcher Struktur die erhaltenen Daten dauerhaft abgelegt werden könnten (*vgl. Kap. 5*). Damit stand nun das eigentliche Korpusmodell im Mittelpunkt.

5. Korpusmodell und Datenorganisation

Parallel zur technischen Entwicklung der Scraping-Pipeline musste festgelegt werden, in welcher Form die gewonnenen Daten abgelegt werden sollten.

Für das Projekt war dabei nicht nur entscheidend, dass Texte lokal gespeichert werden, sondern dass ihre Herkunft, Struktur und Einbettung auch später noch nachvollziehbar bleiben. Deshalb wurde der Korpus als mehrstufiges Datenmodell organisiert.

Die grundlegende Ordnung folgt direkt der Quelle: **Magazin** → **Ausgabe** → **Artikel**.

Diese Struktur ist im realen Korpusbestand unter `data/zxpress/...` klar sichtbar. Sie erlaubt eine saubere Trennung der Metadatenebenen und macht es möglich, einzelne Teile des Bestands unabhängig voneinander zu prüfen, zu ergänzen oder auszuwerten.

Im regulären Fall enthält ein Magazinordner `magazine.json`, `listing.json` und den Unterordner `issues/`.

Ein Beispiel auf Magazinebene `data/zxpress/magazines/Born Dead/magazine.json` zeigt, welche Art von Kontextinformationen dabei bewusst erhalten wurde:

```
{
  "magazine_name": "Born Dead",
  "magazine_url": "https://zxpress.ru/issue.php?id=31",
  "form": "Газета",
  "city_country": "Самара (Россия)",
  "years_human": "октябрь 1998 – июль 2000",
  "years_iso": {
    "start": "1998-10-01",
    "end": "2000-07-01"
  },
  "issues_count": 17,
  "magazine_id": 31,
  "language": "ru",
  "rights": "Использование материалов сайта разрешено только при указании
обратной ссылки",
  "source": "ZXPRESS (https://zxpress.ru)"
}
```

Diese Datei zeigt, dass bereits auf Magazinebene nicht nur technische Identifikatoren, sondern auch Kontextinformationen gespeichert wurden: Name, URL, Publikationsform, Ort, Zeitraum, Sprache, Rechteinweis und Quelle. Das ist für die spätere wissenschaftliche Nutzung zentral, weil der Korpus neben den Texten auch Provenienzdaten bewahrt.

Typischerweise wurde für jede Ausgabe ein eigener Ordner im Format `<issue_label>_<issue_date_iso>` angelegt. Dort liegen `issue.json`, `listing.json` und der Artikelordner.

Auf Ausgabenebene wird dieselbe Logik in verdichteter Form fortgeführt unter `data/zxpress/magazines/Born Dead/issues/0A_1999-06-02/issue.json`:

```
{
  "issue_label": "0A",
  "issue_date_human": "02 июня 1999",
  "issue_date_iso": "1999-06-02",
  "articles_count": 9,
  "issue_slug": "0A_1999-06-02",
  "magazine_id": 31,
  "magazine_name": "Born Dead",
  "language": "ru",
  "city": "Самара",
  "country": "Россия"
}
```

Hier zeigt sich die doppelte Datumslogik des Projekts: Einerseits bleibt die menschenlesbare Datumsform erhalten, andererseits wird das Datum in ISO-Form standardisiert. Diese Kombination ist praktisch, weil sie sowohl Quellennähe als auch maschinelle Weiterverarbeitung ermöglicht.

Auf Artekelebene bestand die Zielstruktur darin, dass jeder Text einen eigenen Unterordner erhielt, dessen Name Reihenfolge, Artikel-ID und einen bereinigten Slug kombiniert. In diesem Ordner liegen `meta.json` und `text.txt`.

Auf Artekelebene meta.json zeigt sich die Trennung von Text und Kontextdaten besonders deutlich:

```
{
  "magazine_id": 31,
  "magazine_name": "Born Dead",
  "city": "Самара",
  "country": "Россия",
  "issue_label": "0A",
  "issue_date_iso": "1999-06-02",
  "order": 1,
  "article_id": 1289,
  "title_link": "Coding - Как кодить оптимально: генерация синус-таблицы, вывод атрибутного черно/белого спрайта.",
  "title_h1": "Born Dead 0A - Coding - Как кодить оптимально: генерация синус-таблицы, вывод атрибутного черно/белого спрайта.",
  "print_url": "https://zxpress.ru/print.php?id=1289",
  "article_url": "https://zxpress.ru/article.php?id=1289",
  "fetched_at": "2025-09-23T19:36:14Z"
}
```

Die Trennung zwischen text.txt und meta.json ist methodisch sinnvoll, weil Rohtext und Kontextdaten nicht vermischt werden. Dadurch bleiben die Texte direkt lesbar, während ihre Einbettung in Magazin, Ausgabe, Ort, Datum und Quelle separat dokumentiert wird.

Zusätzlich zum hierarchischen Korpus wurde eine flache Registersicht aufgebaut.

Diese entsteht durch build_catalog.py, das aus magazine.json, issue.json und articles/*/meta.json drei CSV-Dateien erzeugt: magazines.csv, issues.csv und articles.csv.

Diese Registersicht dient der Übersicht, der Kontrolle und teilweise auch der Vorbereitung weiterer Verarbeitungsschritte. Zur besseren Übersicht und Demonstration beziehen sich die übergebenen CSV-Dateien auf den kleineren Testbestand data_small/zxpress, konkret auf das Magazin „Z80“. Durch Pfadänderung kann die Methode leicht an das ganze Korpus angewendet werden.

magazines.csv enthält in diesem Testfall nur einen Eintrag:

magazine_id	magazine_title
Z80	#Z80

issues.csv bildet die Ausgaben des Testbestands ab:

magazine_id	issue_id	issue_label	issue_date_iso	language	form
Z80	01_1998-05-07	01	1998-05-07	ru	
Z80	02_1998-08-06	02	1998-08-06	ru	
Z80	03_1998-08-23	03	1998-08-23	ru	

articles.csv führt dies bis auf Articlebene weiter und enthält unter anderem den Pfad zum lokalen Textbestand:

magazine_id	issue_id	article_id	title	article_url	issue_date_iso	language	form	filename	content_path
Z80	01_1998-05-07	01_18_68_Bc	01_18_68_Bc	https://zxpress.ru/article.php?id=1868	1998-05-07	ru			/.../text.txt

Diese Registerdateien belegen, dass der Korpus nicht nur als Dateibaum, sondern auch in reduzierter tabellarischer Form gedacht wurde. Gerade für übersichtliche Stichproben, kleinere Tests und spätere Indexierungsfragen war diese zusätzliche Sicht nützlich.

Insgesamt wurde das Korpusmodell damit so aufgebaut, dass es zwei Anforderungen gleichzeitig erfüllt: Es bleibt nah an der Struktur der Quelle und ist zugleich technisch weiterverarbeitbar.

Mit der Festlegung dieser Struktur war der Korpus aber noch nicht automatisch belastbar (*vgl. Kap. 6-7*). Im nächsten Schritt musste geprüft werden, wo Metadaten fehlten, welche Sonderfälle vorlagen und wie der Bestand systematisch validiert und bereinigt werden konnte.

6. Validierung, Bereinigung und Sonderfälle

Mit dem Übergang zu einer modularen Pipeline wurde die Korpusgewinnung nicht mehr als bloßer Scraping-Vorgang behandelt. Spätestens in dieser Phase wurde deutlich, dass der Aufbau eines belastbaren Korpus nicht mit dem Herunterladen von Texten endet. Die Website lieferte zwar eine große Menge an Material, aber die Daten erwiesen sich in mehrfacher Hinsicht als uneinheitlich: Datierungen fehlten oder lagen nur in freier Form vor, einzelne Magazine besaßen keine erschließbaren Ausgaben, Verzeichnisstrukturen konnten voneinander abweichen, und in Einzelfällen traten doppelte oder widersprüchliche Ausgabeneinheiten auf. Daraus ergab sich die Notwendigkeit, den Korpus nicht nur zu erzeugen, sondern systematisch zu prüfen, zu korrigieren und in Sonderfällen gezielt zu reparieren.

6.1 Automatisierte Validierung

Die zentrale Rolle in dieser Phase übernahm `validate_corpus.py`. Das Skript prüft die Struktur eines Magazinordners systematisch auf drei Ebenen: Magazinebene, Ausgabenebene und Artikelebene. Dabei wird kontrolliert, ob die erwarteten Dateien vorhanden sind. Es wird auch kontrolliert, ob wesentliche Metadatenfelder konsistent gesetzt wurden.

Für jede Ausgabe werden `issue.json`, `listing.json` und der Artikelordner berücksichtigt. Auf Artikelebene werden insbesondere `meta.json` und `text.txt` geprüft. Hinzu kommt die Kontrolle von Feldwerten wie `issue_label`, `issue_date_iso`, `article_id`, `title_link` und `print_url`.

Methodisch wichtig ist, dass das Skript nicht jede Unregelmäßigkeit als harten Fehler behandelt. Für problematische Fälle arbeitet es mit Warnungen. Das gilt vor allem für unsichere Datierungen.

Der Mechanismus sieht so aus:

```
elif is_placeholder_date(date_iso):
    warnings.append(f"[Issue #{label}] unbekanntes Datum → Platzhalter  
'0000-01-01'")
```

Damit wird ein Grundsatz sichtbar: Fehlende Informationen werden nicht einfach korrigiert, sondern als Unsicherheiten markiert. Ein nicht gesichertes Datum bleibt als Platzhalter sichtbar und wird nicht durch eine falsche Angabe ersetzt.

Die Validierungsberichte wurden magazinweise unter `logs/validation/` gespeichert.

Ein Beispiel liefert `validate_47th_Byte_20250923_212149.txt`:

```
Magazin: 47th Byte

Validierung: OK - 47th Byte
(mit Warnungen)
- [Issue #01] unbekanntes Datum → Platzhalter '0000-01-01'
→ Issues: 1 | Artikel gesamt: 3
```

Dieser Logeintrag zeigt die Grundlogik sehr klar: Die Struktur kann formal in Ordnung sein, obwohl einzelne Metadaten unsicher bleiben.

Validierung bedeutete im Projekt also nicht einfach nur „fehlerfrei“ oder „fehlerhaft“, sondern eher eine abgestufte Bewertung des realen Bestands.

6.2 Metadaten-Patching und Normalisierung

Der zweite wichtige Bestandteil dieser Phase ist `patch_metadata_light.py`.

Während `validate_corpus.py` Probleme sichtbar macht, dient das Patch-Skript dazu, Metadaten systematisch zu ergänzen und in einzelnen Punkten zu korrigieren. Dazu gehören unter anderem:

- Ergänzung oder Rekonstruktion von `magazine_id`
- Vereinheitlichung von `issues_count`
- Setzen von Standards für `language`, `rights` und `source`
- Berechnung von `years_iso` aus menschenlesbaren Zeitangaben
- Ableitung oder Korrektur von `issue_date_iso`
- Ergänzung von `issue_slug`
- Hinzufügen von `order`, `issue_label` und `short_slug` in `listing.json`
- Neuaufbau des magazinweiten `listing.json`

Besonders aufschlussreich ist dabei der Umgang mit unvollständigen Datierungen.

Wenn eine Ausgabe kein valides Datum besitzt oder nur den Platzhalter 0000-01-01 im verarbeiteten Bestand trägt, versucht das Skript zunächst, das Datum aus dem Ordernamen zu ziehen. Wenn auch das nicht möglich ist, greift es auf den Startzeitraum des Magazins zurück und markiert den Vorgang ausdrücklich als abgeleitet. Dieser Mechanismus zeigt, dass das Projekt nicht einfach nur fehlende Daten auffüllt, sondern den Unterschied zwischen Quelldatum und inferierter Information sichtbar hält.

Neben Datumsfragen betrifft die Normalisierung auch Dateinamen und Slugs.

Die Funktion `slugify(...)` reduziert problematische Zeichen auf dateisystem-taugliche Formen, während der Originaltitel in den Metadaten erhalten bleibt. Diese Trennung ist nötig, weil sie technische Stabilität ermöglicht, ohne die Überlieferungsform des Materials unnötig zu reduzieren.

Bei einem Bestand mit kyrillischen Titeln, Sonderzeichen und transliterierten Mischformen war das eine praktische Voraussetzung für eine stabile Ordnerstruktur.

6.3 Magazinweiser Überblick

Diese Prüf- und Patch-Logik wurde ergänzt durch `audit_corpus.py`.

Dieses Skript erzeugt eine magazinweise Übersicht, in der Korpusstruktur und Validierungsstand zusammengeführt werden. Es zählt für jedes Magazin die Anzahl der Ausgaben und Artikel, prüft, ob Magazin-Listings vorhanden sind, und verbindet diese Informationen mit dem jeweils neuesten Validierungslog.

Das Ergebnis kann als CSV exportiert werden und liegt im Projekt als `logs/audit_summary.csv` vor.

Diese Datei ist besonders nützlich, weil sie einerseits den Umfang des Korpus dokumentiert und andererseits Problemfälle sichtbar macht.

Bereits die ersten Zeilen zeigen, wie unterschiedlich die Magazine im Bestand ausgeprägt sind: Es finden sich Titel mit wenigen Artikeln ebenso wie sehr umfangreiche Reihen. Gleichzeitig werden Magazine ohne Ausgaben und Sonderfälle gesondert sichtbar.

So erscheinen etwa `BBSTOP`, `Batva`, `CAFe_2003` oder `Enigma_Tape_Magazine` mit 0 Ausgaben und 0 Artikeln, während Magazine wie `Nicron`, `Body`, `On-Line` oder `ZX Pilot` sehr große Bestände aufweisen. Der Auditbericht macht damit den Korpus nicht nur technisch kontrollierbar, sondern auch in seiner internen Struktur überschaubar.

Für die Projektdokumentation ist außerdem wichtig, dass `audit_summary.csv` Fälle mit Warnstatus nicht mit strukturellem Scheitern gleichsetzt.

Ein Eintrag wie `OK+WARN` bedeutet im Regelfall, dass die Verzeichnisstruktur grundsätzlich verwendbar ist, aber es können Platzhalterdaten oder andere Auffälligkeiten vorliegen. Das entspricht dem bereits in `validate_corpus.py` sichtbaren Grundprinzip einer abgestuften Qualitätsbewertung.

6.4 Leere Magazine und der Umgang mit Nullbeständen

Ein wiederkehrender Sonderfall im Korpus waren Magazine, die im Katalog der Website vorhanden waren, für die jedoch keine auswertbaren Ausgaben gewonnen werden konnten.

In `logs/health/magazines_empty.log` sind 29 solcher Magazine dokumentiert, darunter `BBSTOP`, `Batva`, `Fantasy`, `Scene`, `Woot`, `Xa-XaFon` und `ZX_Land`. Diese Fälle wurden nicht aus dem Bestand entfernt, sondern bewusst als leere oder unvollständige Magazine behandelt.

Das ist methodisch sinnvoll, weil diese Magazine zur Struktur der Quelle gehören, auch wenn sie für den eigentlichen Textkorpus keinen Inhalt liefern. Würde man sie einfach löschen, produziere man ein verzerrtes Bild des Ausgangsbestands.

Das Projekt unterscheidet damit sinnvoll zwischen „kein Bestand vorhanden“ und „Bestand vorhanden, aber ohne erschließbaren Inhalt“.

6.5 Health-Checks und ihre Grenzen

Für die globale Strukturkontrolle wurde zusätzlich `health_zxpress.sh` eingesetzt.

Dieses Shell-Skript prüft, ob auf Ausgabeebene `issue.json`, `listing.json` und `articles/` vorhanden sind und ob auf Artikelebene `meta.json` und `text.txt` fehlen.

Außerdem protokolliert es Platzhalterdaten und leere Magazine. Für die eigentliche Fehlerdetektion ist dieses Skript gut geeignet.

Die zugehörigen Logs unter `logs/health` zeigen beispielsweise, dass `issues_missing.log` und `articles_missing.log` leer sind, also keine fehlenden Strukturdateien mehr gefunden wurden. Auch `magazines_empty.log` liefert eine klare Liste leerer Magazine.

Gleichzeitig muss die Auswertung von `summary.txt` vorsichtig erfolgen.

Dort erscheinen zwar `Magazines: 291`, aber `Issues: 0` und `Articles: 0`. Diese beiden Nullwerte können nicht den tatsächlichen Korpusbestand wiedergeben, da sie durch `audit_summary.csv`, einzelne Validierungslogs und den späteren Full-Healthcheck klar widerlegt werden (*vgl. Kap. 7.4; 8:8*). Die manuelle Überprüfung des Korpus und Abgleich mit der Originalquelle durch den Forscher wäre an dieser Stelle sehr zeitaufwendig.

Diese Beobachtung zeigt, dass im Projekt nicht nur mit Werkzeugen gearbeitet, sondern deren Ergebnisse auch kritisch geprüft wurden. Es sollte deutlich zeigen, welche Zahlen belastbar sind und welche nur im jeweiligen technischen Kontext sinnvoll interpretiert werden können.

6.6 Reparatur von Sonderfällen

Ein wichtiger konkreter Sonderfall war das Magazin „`Psychoz`“. In `audit_summary.csv` erscheint es mit `issues_missing_articles_dir = 1` und dem Status `OK+WARN`, was bereits darauf hinweist, dass hier eine besondere Inkonsistenz vorlag.

Ein Beispielskript `repair_psychoz_12.py` führt zwei voneinander abweichende Pfade zusammen:

- `data/zxpress/magazines/Psychoz/issues/12_0000-01-01`
- `data/zxpress/magazines/Psychoz/issues/12/12_0000-01-01`

Das Skript lädt die beiden `listing.json`-Dateien, vergibt die Reihenfolge neu, setzt durchgängig `issue_label = "12"`, ergänzt fehlende `short_slug`-Werte und aktualisiert anschließend `issue.json` der Ausgabe. Danach wird der redundante zweite Pfad entfernt.

Der Kern dieses schnellen Reparaturschritts besteht in der Zusammenführung zweier widersprüchlicher Verzeichnisäste zu einer konsistenten Ausgabe.

6.7 Nachladen fehlender Artikel

Ein weiterer Teil der Bereinigungslogik bestand im gezielten Nachladen fehlender Artikel.

`fetch_articles_light.py` ist nicht nur für den Erstabruf zuständig, sondern besitzt mit `--retry-missing` auch einen expliziten Mechanismus, um fehlende Texte oder Metadaten nachträglich zu ergänzen.

Also: wenn ein Heft zwar im Listing erfasst wurde, aber einzelne Artikelordner oder Dateien fehlen, können diese gezielt erneut geladen werden, ohne den gesamten Korpus neu aufzubauen.

Gerade im Zusammenspiel mit den Validierungslogs ist das sinnvoll.

Erst die Validierung macht sichtbar, wo Lücken bestehen; danach kann `fetch_articles_light.py` selektiv eingesetzt werden.

Diese Trennung von Diagnose und Nachbesserung ist ein wesentlicher Fortschritt gegenüber einem monolithischen Scraper, der bei jedem Problem nur einen vollständigen Neudurchlauf zulassen würde. Damit kann der Forscher einige Zeit bei der Korpusbereinigung sparen.

6.8 Zwischenergebnis

Die Validierungs- und Bereinigungsphase markierte den Punkt, an dem aus einer funktionierenden Scraping-Pipeline eine kontrollierte Korpusproduktion wurde.

Die Kombination aus `validate_corpus.py`, `patch_metadata_light.py`, `audit_corpus.py`, `health_zxpress.sh`, gezieltem Nachladen und Sonderfallskripten wie `repair_psychoz_12.py` zeigt, dass der Korpus nicht nur gesammelt, sondern systematisch gepflegt wurde.

7. Endvalidierung und Abschluss der Korpusstruktur

Dieses Kapitel fasst den erreichten Endzustand des Korpus zusammen. Im Mittelpunkt steht nun nicht mehr das einzelne Werkzeug, sondern die Frage, wie stabil und vollständig der Bestand nach diesen Arbeitsschritten tatsächlich vorliegt.

Nach Validierung, Metadaten-Patching und Behandlung einzelner Sonderfälle stellte sich die Frage, ob der Korpus in einer Form vorliegt, die für die nächsten Projektschritte tatsächlich tragfähig ist.

Das Ziel der Endvalidierung war dreifach. Erstens sollte überprüft werden, ob die Verzeichnisstruktur formal konsistent ist, also ob Magazine, Ausgaben und Artikel in der erwarteten Form vorliegen. Zweitens sollte geklärt werden, welche Arten von Problemen nach den Bereinigungsschritten noch übrig blieben: strukturelle Lücken, fehlende Dateien, Platzhalterdaten oder leere Magazine. Drittens ging es um eine belastbare Größenbestimmung des Korpus, weil diese später sowohl für die Indexierung relevant ist.

Wichtig ist dabei, dass „abgeschlossen“ hier nicht bedeutet, dass alle Quelldaten perfekt sind. Die Website selbst bleibt eine historisch gewachsene und heterogene Quelle. Endvalidierung meint in diesem Zusammenhang, dass der lokale Korpusbestand in sich stabil genug ist, um darauf weitere Verarbeitung aufzubauen.

7.1 Strukturelle Vollständigkeit

Für die strukturelle Integrität des Korpus sind die Health-Logs besonders aussagekräftig.

In `logs/health/issues_missing.log` und `logs/health/articles_missing.log` finden sich keine Einträge. Das bedeutet, dass in diesem Prüflauf keine fehlenden `issue.json`, `listing.json`, `articles/`, `meta.json` oder `text.txt` mehr registriert wurden.

Für die formale Vollständigkeit des Korpus ist das ein starker Befund: Nach den Bereinigungs- und Nachladeprozessen lagen die erwarteten Dateien offenbar durchgängig vor.

Gleichzeitig muss die Auswertung von `logs/health/summary.txt` differenziert erfolgen.

Die dort genannten Werte

- Magazines: 291
- Issues: 0
- Articles: 0

können offensichtlich nicht als reale Korpusgröße gelesen werden. Sie widersprechen sowohl `audit_summary.csv` als auch den Validierungslogs (auf Magazin-Ebene) und dem späteren Full-Healthcheck.

Es ist deswegen festzuhalten: Das Health-Skript war für die Erkennung fehlender Dateien und leerer Magazine nützlich, seine Summenzeilen zu Issues und Artikeln sind in dieser Fassung jedoch wenig belastbar.

Die Strukturprüfung bleibt trotzdem verwertbar, weil die eigentlichen Fehlerprotokolle leer sind.

7.2 Leere Magazine als dokumentierter Sonderfall

Ein Restbestand von Sonderfällen betrifft Magazine, die zwar im Ausgangskatalog vorkommen, aber keine auswertbaren Ausgaben enthalten.

In `logs/health/magazines_empty.log` (vgl. Kap. 6.3) sind 29 solcher Magazine dokumentiert. Auch `audit_summary.csv` bestätigt diese Fälle, da die betreffenden Magazine dort mit 0 Ausgaben und 0 Artikeln geführt werden.

Diese Magazine bleiben daher im Verzeichnisbestand und in den Auswertungen sichtbar, werden aber als leere Einheiten behandelt. Methodisch ist das sinnvoll, um zu zeigen, dass der Korpus damit nicht künstlich bereinigt wird, indem problematische oder leere Ausgangsobjekte einfach verschwinden.

7.3 Platzhalterdaten und verbleibende Unsicherheiten

Komplexer ist die Lage bei den Datierungen.

In den magazinweisen Validierungslogs und im Auditbericht erscheinen zahlreiche Magazine mit Status `OK+WARN`. Ein typischer Warnfall ist ein Platzhalterdatum `0000-01-01`, wenn für eine Ausgabe kein sicheres Datum rekonstruiert werden konnte.

Das Beispiel „47th Byte“ (vgl. Kap. 6.1) zeigte diesen Fall ausdrücklich.

Diese Art von Unsicherheit wurde im Laufe der Arbeit nicht als strukturelles Scheitern gewertet. Der Korpus ist formal stabil, aber nicht alle Metadaten besitzen dieselbe Präzision. Genau deshalb muss zwischen struktureller Vollständigkeit und inhaltlicher bzw. metadatenbezogener Perfektion unterschieden werden.

Aus den vorliegenden Daten ergibt sich zudem, dass `placeholders_0000-01-01.log` in `logs/health/` leer ist, während andere Dateien weiterhin Warnfälle mit Platzhalterdaten belegen. Für die Endbewertung heißt das: Die Health-Datei kann hier nicht als alleinige Wahrheit gelten. Nützlicher ist die Gesamtschau der logs und skripts.

7.4 Bestandszahlen des Korpus

Für die quantitativen Abschlussangaben ist `audit_summary.csv` die belastbarere Quelle als `summary.txt` aus den Health-Logs.

Sie dokumentiert magazinweise Anzahl der Ausgaben und Artikel sowie den jeweiligen Validierungsstatus. In Verbindung mit dem späteren Full-Healthcheck ergibt sich ein klareres Bild des Projektstands (vgl. Kap. 8.8).

Der Full-Healthcheck nennt folgende Zahlen:

- 291 Magazine
- 1.875 Issues
- 17.148 Artikeltexte als `text.txt`
- 17.148 `meta.json`
- 17.148 sichtbare Indexdokumente

Diese Werte passen grundsätzlich zu dem Eindruck aus `audit_summary.csv`, wo große Bestände wie Nicron, Body, On-Line, ZX-News oder Maximum ebenso sichtbar werden wie viele kleinere Magazine und die Gruppe der 29 leeren Magazine.

Kleinere Differenzen zwischen verschiedenen Zählständen im Projektverlauf sind dabei plausibel, etwa weil einzelne Reparaturen, Nachladungen oder Audits zu unterschiedlichen Zeitpunkten ausgeführt wurden.

Für die Abgabefassung sind deshalb die späteren Zahlen des Healthchecks zu bevorzugen und frühere Zwischenstände nur als Arbeitsstände zu behandeln.

Der notwendige Aspekt der Endvalidierung betrifft nicht nur den Dateibestand, sondern auch die Anschlussfähigkeit an die spätere Suchinfrastruktur.

Der Überprüfung weist ausdrücklich darauf hin, dass die Zahl der gespeicherten Artikeldateien und die Zahl der sichtbaren Indexdokumente übereinstimmen. Damit ist ein zentraler Übergangspunkt des Projekts markiert: Der Korpus liegt im Dateisystem vor und ist in einer Form vorhanden, die sich in den Suchindex übertragen ließ.

Der erreichte Zustand des Korpus hatte direkte Folgen für die kommenden Arbeitsschritte.

Auf dieser Basis konnte ein Index aufgebaut werden, ohne dass grundlegende Strukturfehler zwischen Magazin-, Ausgaben- und Artikelebene den Prozess blockierten. Die nötigen Metadaten stehen bereit, um später Filter, Kontextausgabe und magazinbezogene Abfragen zu unterstützen.

Für das Projekt insgesamt markiert diese Endvalidierung daher einen echten Abschluss des Korpusaufbaues. Im nächsten Schritt soll die Überführung des stabilisierten Bestands in einen Suchindex und der Aufbau eines Rechercheprototyps erfolgen.

8. Lucene-Indexierung und Entwicklung des Suchprototyps

Nach dem Aufbau und der Stabilisierung des Korpus verschob sich der Schwerpunkt des Projekts auf die Suchbarkeit des Bestands.

Nun sollten die gewonnenen Texte und Metadaten in eine Form überführt werden, in der Volltextsuche, Metadatenfilterung und Kontextanzeige praktisch möglich werden.

Für diesen Schritt wurde Lucene als Suchkern gewählt, in der Python-Anbindung über `PyLucene`.

Diese Entscheidung war im Projektkontext sinnvoll. Zum einen bringt Lucene zentrale Funktionen wie Volltextabfragen, Boolesche Operatoren, Feldsuche und numerische Filter bereits mit. Zum anderen passte die Python-Anbindung gut zur bestehenden Projektstruktur, da der Korpusaufbau, die Metadatenverarbeitung und die Validierung bereits weitgehend in Python umgesetzt worden waren. Auf diese Weise konnte ein lokaler, kontrollierbarer und erweiterbarer Suchprototyp entwickelt werden, ohne den bisherigen Workflow in ein anderes technisches Umfeld überführen zu müssen.

8.1 Übergang vom Korpus zum Suchindex

Die Verbindung zwischen Korpus und Suchindex wurde in `scripts/TextSearch/Indexer.py` umgesetzt.

Dieses Skript liest den Korpus entweder direkt aus dem hierarchischen Verzeichnisbaum oder alternativ aus einem flachen CSV-Katalog ein. Beide Wege sind im Code manuell vorgesehen. Ohne Katalog arbeitet der Indexer auf dem Magazin–Issue–Artikel-Baum, mit Katalog verwendet er `articles.csv` als vereinfachte Zwischensicht.

Diese Doppellogik war sinnvoll, weil sie zwei Arbeitsweisen unterstützte: zum einen die Indexierung des vollständigen Korpusbestands unter `data/zxpress/...`, zum anderen kleinere Testläufe auf Basis reduzierter Katalogdaten wie `data_small`. Gerade für Entwicklung, Fehlersuche und schnellere Experimente war diese zweite Möglichkeit nützlich.

Maßgeblich ist im Indexer die Funktion `_iter_legacy_articles()`, die Metadaten aus `magazine.json`, `issue.json`, `meta.json` und `text.txt` zusammenzieht.

Eingelesen werden also nicht nur die Volltexte selbst, sondern auch zentrale Kontextfelder wie Magazinname, Publikationsform, Sprache, Datum, URL und Ausgabenlabel.

8.2 Indexierte Felder

Beim Schreiben der Lucene-Dokumente wurde bewusst zwischen Volltextfeldern, Filterfeldern und Anzeige-/Hilfsfeldern unterschieden.

Das ist im Kern des Indexers direkt sichtbar:

```
doc = Document()
doc.add(TextField("content", content, Field.Store.YES))
doc.add(TextField("title", title, Field.Store.YES))

doc.add(StringField("magazine", mag, Field.Store.YES))
doc.add(StringField("form", form, Field.Store.YES))
doc.add(StringField("language", lang, Field.Store.YES))

doc.add(StoredField("filename", fname))
if dateiso:
    doc.add(StoredField("issue_date_iso", dateiso))
if url:
    doc.add(StoredField("article_url", url))
```

Hier zeigt sich die grundlegende Architektur des Suchprototyps.

content und title wurden als eigentliche Suchfelder mit `TextField` abgelegt.

Die Felder magazine, form und language wurden dagegen als `StringField` gespeichert, also für exakte Filterung statt für tokenisierte Volltextsuche.

Diese Unterscheidung ist funktional wichtig. Die Suche nach einem Wort im Artikeltext folgt einer anderen Logik als die gezielte Einschränkung auf ein bestimmtes Magazin oder eine bestimmte Publikationsform.

Zusätzlich wurde das Ausgabedatum nicht nur als `ISO-String` gespeichert, sondern auch numerisch für Zeitbereichsfilter aufbereitet:

```
epoch = iso_to_epoch_ms(dateiso)
if epoch is not None:
    doc.add(LongPoint("issue_date_epoch_ms", epoch))
    doc.add(StoredField("issue_date_epoch_ms", epoch))
```

Damit wurde bei der Indexierung berücksichtigt, dass spätere Abfragen auch zeitlich eingeschränkt werden sollten.

8.3 Frühe Suchtests und schrittweiser Ausbau

Die Suchfunktion entstand nicht sofort in ihrer späteren Form.

Am Anfang wurde zunächst mit sehr einfachen Abfragen experimentiert: einzelne Begriffe im Feld content, direkte Konsolenausgabe und kleine Stichproben.

Ein Beispiel dafür findet sich in `textsuche_klein.py`, wo eine reduzierte Lucene-Abfrage mit anschließender KWIC-Ausgabe getestet wurde.

Erst im weiteren Verlauf wurde die Suche schrittweise erweitert. Dazu gehörten:

- die Einbeziehung des Titelfelds zusätzlich zum Volltext
- die Speicherung von Magazin-, Sprach- und Formfeldern
- die Umrechnung von Datumsangaben in numerische Werte für Range-Queries
- die Entwicklung einer interaktiven Kommandozeilenabfrage
- Anschließend die spätere Ergänzung von KWIC und Fallback-Snippets

8.4 Der lokale Index im Ordner `index_dir/`

Das Ergebnis dieses Schritts ist der lokale Lucene-Index im Ordner `index_dir/`. Die dort sichtbaren Dateien wie `_a.cfe`, `_a.cfs`, `_a.si`, `segments_2` oder `write.lock` sind keine direkt lesbaren Textdaten, sondern die interne Persistenzstruktur des Index im Lucene.

Für das Projekt ist diese Trennung methodisch wichtig. Die eigentlichen Quellen und Metadaten bleiben im JSON-/TXT-basierten Korpus unter `data/zxpress/...` erhalten.

`index_dir/` enthält dagegen nur die abgeleitete Suchrepräsentation. Dadurch bleibt der Korpus selbst unabhängig von der Suchmaschine lesbar und nachvollziehbar, während der Index bei Bedarf neu erzeugt werden kann.

8.5 Entwicklung des Suchprototyps in `Searcher.py`

Auf dem aufgebauten Index setzte `scripts/TextSearch/Searcher.py` auf.

Dieses Skript bildet den eigentlichen Rechercheprototyp und ist interaktiv und auch per Kommandozeile nutzbar.

Die Suchlogik ist als Kombination aus Volltext- und Metadatenabfrage umgesetzt.

Wenn ein Suchtext eingegeben wird, wird dieser parallel auf content und title angesetzt:

```
qp_content = QueryParser("content", analyzer)
qp_title   = QueryParser("title", analyzer)
qc = qp_content.parse(qtext)
qt = qp_title.parse(qtext)
inner = BooleanQuery.Builder()
inner.add(qc, BooleanClause.Occur.SHOULD)
inner.add(qt, BooleanClause.Occur.SHOULD)
q = inner.build()
```

Die Abfrage beschränkt sich nicht auf den Fließtext, sondern berücksichtigt zugleich Titelinformationen. Das soll die praktische Nutzbarkeit deutlich erhöhen, weil viele Suchanfragen in einem Magazinbestand nicht nur den Artikelinhalt, sondern auch Rubriken und Überschriften betreffen.

Darauf werden die Metadatenfilter aufgesetzt.

```
Magazin, Form und Sprache werden als exakte Term-Filter eingebunden,
Zeiträume über ein LongPoint-Range-Query:

if magazine:
    b.add(TermQuery(Term("magazine", magazine)),
          BooleanClause.Occur.FILTER)

if lang:
    b.add(TermQuery(Term("language", lang)), BooleanClause.Occur.FILTER)

if year_from or year_to:
    start = iso_to_epoch_ms(f"{year_from}-01-01") if year_from else None
    end   = iso_to_epoch_ms(f"{year_to}-12-31")   if year_to   else None
    b.add(LongPoint.newRangeQuery("issue_date_epoch_ms", start, end),
          BooleanClause.Occur.FILTER)
```

Der Suchprototyp ging damit bereits deutlich über eine reine Textsuche hinaus.

Bei den ersten Tests tauchten konkrete Praxisprobleme auf:

- Russische Begriffe wurden nicht immer gefunden (z.B. Unterschiede zwischen „поздравляю“ und „поздравления“, de: gratuliere und gratulationen)
- Uneindeutige Magazin-Namen führten zu Fehltreffern
- Platzhalterdaten (0000-01-01) konnten Filter verfälschen.

Beispiel für folgende Outputs:

```
Suchanfrage: поздравляю #de: gratuliere
Magazin: z80
Treffer: 0
auch wenn im Korpus die Texte mit diesem Wort vorhanden sind.
```

Zur Lösung wurden im Laufe der Tests schrittweise weitere kleine Bausteine ergänzt:

- Wildcards und Fuzzy-Suche: Luciene Unterstützung von * in Queries (z.B. поздрав*).

```
qp = QueryParser("content", analyzer)
query = qp.parse("поздрав*") # findet "поздравляю", "поздравления" usw.
```

- Magazin-Normalisierung: Case-insensitive Matching und Vorschlagslisten bei Mehrdeutigkeiten.

```
mapping = {"журнал": "Журнал", "газета": "Газета"} #ru: журнал, de:
magazin; ru: газета, de: zeitschrift
form = mapping.get(user_form.lower(), user_form)
```

Ein sinnvoller Aspekt von `Searcher.py` ist, dass dort auch mögliche praktischen Bedienungsprobleme berücksichtigt wurden.

Wird das Skript ohne Parameter aufgerufen, fragt es Suchtext, Magazin, Form, Sprache, Jahre und KWIC-Einstellungen interaktiv per Prompt ab. Damit entstand eine kleine REPL-artige Suchumgebung, die für Testläufe und manuelle Exploration nützlich war.

Zusätzlich wurde ein Resolver für Magazinnamen eingebaut. Dieser unterscheidet exakte Treffer, Präfixsuche und Teiltreffer. Falls eine Eingabe nicht eindeutig ist, sollten Vorschläge ausgegeben werden. Diese Funktion war vor allem deshalb sinnvoll, da Magazinnamen im Bestand teilweise ähnlich oder mehrdeutig waren und eine reine Exaktabfrage zu unpraktisch gewesen wäre.

8.6 KWIC als Ausgabeverfahren

Bereits in der konzeptuellen Phase war klar, dass Suchergebnisse nach Möglichkeit im Sinne von Key Words in Context ausgegeben werden sollten.

In `Searcher.py` wurde dafür eine eigene `kwic()`-Funktion implementiert:

```
def kwic(txt, term, window=5, max_snips=3):
    words = txt.split()
    term_low = (term or "").lower()
    out = []
    if not term_low:
        return out
    for i, w in enumerate(words):
        if term_low in w.lower():
            s = max(0, i - window); e = min(len(words), i + window + 1)
            out.append(" ".join(words[s:e]))
            if len(out) >= max_snips:
                break
    return out
```

Diese Implementierung arbeitet wortbasiert, liefert mehrere kurze Trefferkontexte und ist unabhängig davon, dass Lucene selbst keine fertige KWIC-Darstellung bereitstellt.

Die Kontextanzeige musste daher im Projekt selbst ergänzt werden. Das ist nicht nur technisch relevant, sondern auch inhaltlich: Die KWIC-Ausgabe schlägt bereits eine Brücke zum späteren FCS-Ziel, da FCS-Treffer typischerweise nicht nur als Trefferzahl, sondern als kontextualisierte Hits präsentiert werden (*vgl. Kap. 9.6; 10.4*).

8.7 Fallback-Strategien und Datenprobleme

Der Suchprototyp wurde außerdem wieder so erweitert, dass er auf reale Datenprobleme reagieren konnte.

Der KWIC-Begriff wird aus der Query abgeleitet, aber für die Snippet-Erzeugung von Wildcards und Sonderzeichen bereinigt.

Wenn die wortbasierte KWIC-Funktion nichts findet, greift ein Zeichenfenster-Fallback:

```
def _fallback_snippet(text: str, needle: str, chars: int = 180):
    if not text or not needle:
        return None
    t1 = text.lower()
    n1 = needle.lower()
    i = t1.find(n1)
    if i == -1:
        return None
```


Diese Ergänzung zeigt, dass die Suchergebnisse nicht nur unter idealen Bedingungen gedacht wurden. Vielmehr wurde versucht, auch dann noch nützliche Kontextausschnitte zu liefern, wenn Tokenisierung, Schreibvarianten oder OCR-nahe Probleme eine direkte KWIC-Zuordnung erschweren.

Dazu gehörten Verwechslungen zwischen kyrillischen und lateinischen Zeichen, uneinheitliche Metadaten und variable Schreibweisen.

8.8 Zusatzskripte als Diagnose- und Prüfwerkzeuge

Neben `Indexer.py` und `Searcher.py` existieren mehrere kleinere Hilfsprogramme unter `scripts/TextSearch/_tools/`, darunter `feldabdeckung.py`, `indexüberprüfen.py`, `textsuche_klein.py` und `full_healthcheck.py`.

Diese Dateien sind keine Hauptkomponenten des Suchsystems, sondern nützliche Diagnose- und Entwicklungswerkzeuge.

- `textsuche_klein.py` diente als reduzierte Testumgebung für erste Suchversuche
- `indexüberprüfen.py` prüfte stichprobenartig, ob in zufällig ausgewählten Indextokumenten wichtige Felder vorhanden sind
- `feldabdeckung.py` zählte die Abdeckung einzelner gespeicherter Felder und sammelte Beispiele fehlender Informationen
- `full_healthcheck.py` verband Korpusprüfung und Indexprüfung in einem gemeinsamen Überblick

Mit diesen Werkzeugen wurde der Index systematisch geprüft.

Besonders nützlich ist `scripts/TextSearch/_tools/full_healthcheck.py`.

Das Skript vergleicht Dateibestand und Lucene-Index direkt miteinander und kontrolliert zusätzlich die Feldabdeckung im Index. Der dokumentierte Output zeigt einen fast vollständigen Gleichstand zwischen Korpus und Suchindex:

Zusätzlich wurde ausdrücklich ausgegeben:

```
Datei/Index Konsistenz: OK
```

Damit lässt sich belegen, dass die Indexierung auf dem vollständigen Korpus ausgeführt und geprüft wurde.

Auch die Feldabdeckung war weitgehend vollständig:

Vollständig vorhanden waren unter anderem `content`, `magazine`, `form`, `language`, `issue_label`, `issue_date_iso`, `title`, `article_url` und `print_url`.

Teilweise fehlten dagegen Felder wie `city`, `country` oder numerische Zeitwerte.

Gleichzeitig zeigt derselbe Output ein auffälliges Problem bei der Epoch-Spanne.

Dort werden Minimal- und Maximalwerte ausgegeben, die auf Daten um 1969/1970 verweisen und damit nicht zu den realen Heftjahren des Korpus passen. Dieser Befund spricht dafür, dass die numerische Datumsumrechnung oder deren spätere Auswertung noch nicht vollständig korrekt war.

Für die grundsätzliche Volltextsuche war dies nicht blockierend, für robuste Zeitfilterung jedoch relevant. Der Healthcheck ist damit einerseits ein Beleg für die weitgehende Stabilität des Indexes, andererseits auch ein Hinweis darauf, dass die Zeitrepräsentation noch nicht in jedem Punkt ausgereift war.

8.9 Technische Schwierigkeiten und Ergebnis der Lucene-Phase

Die Lucene-Phase verlief technisch nicht reibungslos. Zu den technischen Problemen gehörten insbesondere:

- die Notwendigkeit, die JVM explizit mit `lucene.initVM()` zu starten
- Unterschiede zwischen Java-Lucene-Beispielen aus der Dokumentation und dem realen Verhalten von PyLucene
- Probleme mit falschen Python-Umgebungen unter macOS
- PyCharm-Warnungen bei Java-Imports trotz lauffähigem Code
- Die Tatsache, dass die Indexdateien selbst binär und nicht direkt lesbar sind

Hinzu kamen API-Unterschiede in der Dokumentenabfrage.

In Java-Beispielen erscheint häufig der Zugriff `searcher.doc(hit.docID)`, während in PyLucene die gespeicherten Felder tatsächlich über `reader.storedFields().document(hit.doc)` gelesen werden mussten. Solche Unterschiede mussten im Projekt praktisch erarbeitet werden und gehörten zum realen Implementierungsaufwand.

Am Ende dieser Phase lag ein funktionsfähiger lokaler Suchprototyp vor, der auf dem selbst aufgebauten Korpus arbeitete:

- Volltext und Titel wurden gleichzeitig durchsucht.
- Das Filtern war möglich nach Magazin, Jahr, Form und Sprache.
- Eingaben wurden interaktiv entgegennimmt.
- Fallback-Strategien wurden angeboten, wenn z.B. Magazineingaben nicht eindeutig sind.

Ein Beispiel des Outputs bei der einfachen Suche nach Wort “спектрум” (spectrum)

```
===== Einfache Suche =====
Geben Sie die Suchanfrage ein (leer = alle): спектрум #de: spectrum
Magazin (exakt, leer = egal): ZX Magazine
Form (Журнал/Газета): Газета #de: Журнал = Magazin, Газета = Zeitschrift
Jahre: 1995-1997
Treffer: 25
```

Zusammen ergibt sich eine vollwertige Rechercheumgebung für das Korpus.

Der Bestand war in Lucene indexiert, der Index durch mehrere Hilfsskripte kontrolliert, und über Searcher.py konnten Volltextabfragen mit Filterung und KWIC-Ausgabe ausgeführt werden.

Der Korpus war damit nicht nur gesammelt, sondern bereits praktisch recherchierbar. Daraus ergab sich der nächste Schritt, nämlich die Überführung dieser lokalen Suchlogik in einen standardisierten SRU-/FCS-Endpoint (vgl. Kap. 9-10).

9. Erster Versuch eines eigenen FCS-/SRU-Endpoints (scripts/FCS/)

9.1 Spezifikations- und Orientierungsphase

Die aufgebaute Suche muss nicht nur lokal, sondern als standardisierter Suchdienst bereitgestellt werden.

Ziel war nun, den bereits funktionierenden Recherhekern in einen FCS-Endpoint zu überführen, der Anfragen entgegennimmt, Suchergebnisse in XML ausgibt und perspektivisch über einen externen Validator geprüft werden kann.

Diese Protokollschicht sollte zwischen der vorhandenen Suchmaschine und einem externen Client vermitteln. Der Endpoint musste die Treffer finden und diese in einer Form ausgeben, die sich an den FCS- und SRU-Spezifikationen orientiert.

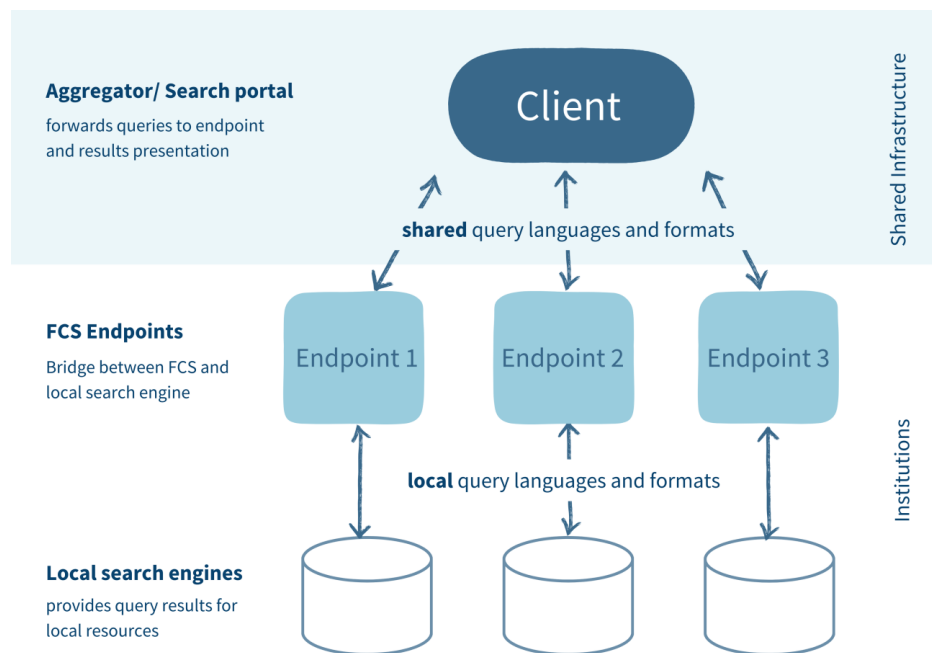


Abb.1 - Kernidee der Verbindung zwischen lokalen Suchmaschinen und dem Suchaggregator von Clarin

CLARIN ist eine europäische Forschungsinfrastruktur für sprachbezogene Daten, Werkzeuge und Dienste.

Ziel der Initiative ist, Sprachressourcen nachhaltig zugänglich und für Forschung in den Geistes- und Kulturwissenschaften nutzbar zu machen. Für Digital Humanities ist CLARIN deshalb relevant, weil dort gleichzeitig Korpora und technische Standards und Werkzeuge bereitgestellt werden.

Innerhalb dieser Infrastruktur ist Federated Content Search (FCS) die Suchschnittstelle, über die Anfragen an unterschiedliche, dezentral betriebene Datenbestände gestellt werden können. Ein FCS-Endpoint bildet dabei die technische Verbindung zwischen einem lokalen Korpus bzw. einer Suchmaschine und dem gesamten Suchsystem. Der FCS Aggregator fungiert als zentrale Suchoberfläche: Er nimmt eine Anfrage entgegen, leitet sie an mehrere angeschlossene Endpoints weiter und bündelt die Ergebnisse in einer gemeinsamen Oberfläche. Dadurch muss nicht jedes Korpus einzeln durchsucht werden.

Für DH-Forschung liegt der Nutzen vor allem darin, dass sprachliche und textuelle Bestände aus unterschiedlichen Institutionen über eine gemeinsame Recherchelogik vergleichbar gemacht werden.

FCS erlaubt damit neben der Volltextsuche die kontextuelle Anzeige von Treffern und die Arbeit mit standardisierten Abfrageformaten wie CQL.

Vor dem eigentlichen Bau des Endpoints wurden zunächst die Anforderungen der CLARIN-Umgebung nachvollzogen unter

<https://www.clarin.eu/content/federated-content-search-clarin-fcs-technical-details>.

Relevante Hilfsmittel dabei sind:

- **FCS Core Specification 2.0**
<https://office.clarin.eu/v/CE-2017-1046-FCS-Specification-v20230426.pdf>
- **FCS Data View Specification 1.0**
https://office.clarin.eu/v/CE-2014-0317-CLARIN_FCS_Specification_DataViews_1_0-v20170613.pdf
- **XML-Schemata und Referenzbeispiele**
<https://github.com/clarin-eric/fcs-misc.git>
- **Der FCS Endpoint Validator**
<https://fcs-validator.data.saw-leipzig.de/>
- Allgemeine SRU-Logik mit explain, searchRetrieve und diagnostischen Responses

Besonders relevant für die Implementierung waren dabei zwei Fragen:

1. Welche Operationen muss ein Endpoint mindestens beherrschen?
2. Wie müssen Treffer als XML strukturiert werden, damit sie als FCS-kompatibel gelten können?

Für die Implementierung des ersten Versuchs waren vor allem zwei Teile der Spezifikation genügend:

- `explain`: beschreibt den Endpoint selbst, also etwa Fähigkeiten, Datenansichten, Sprachen und Ressourcen
- `searchRetrieve`: liefert eigentliche Suchtreffer mit Metadaten und Trefferdarstellung

Zusätzlich wurde die Data-View Logik wichtig, weil Treffer nicht nur als bloße Trefferzahl, sondern nach Möglichkeit mit KWIC-artigem Kontext ausgegeben werden sollten.

9.2 Aufbau des ersten Prototyps

Der erste konkrete Versuch wurde unter `scripts/FCS/` umgesetzt.

Zentrale Dateien dieses Strangs sind:

- `scripts/FCS/fcs_endpoint.py`
- `scripts/FCS/fcs_xml.py`
- `scripts/FCS/cql_parser.py`
- `config/zxpress.yaml`

Diese Kombination zeigt die Grundarchitektur des ersten FCS-Versuchs:

- `fcs_endpoint.py` übernimmt Routing, Anfrageverarbeitung, Lucene Anbindung und Response Steuerung
- `fcs_xml.py` erzeugt die XML-Antworten für `Explain`, `SearchRetrieve` und `Diagnostics`
- `cql_parser.py` dient als Zwischenstufe zwischen SRU-/CQL-Anfrage und interner Lucene-Abfrage
- `zxpress.yaml` weitere Konfigurationsdaten liefern Projektparameter

Der Endpoint wurde als eigener Flask-Server implementiert. Anders als im späteren Ansatz (*vgl. Kap. 10*) wurde die Protokollschicht hier weitgehend selbst gebaut.

Am Anfang der Datei `fcs_endpoint.py` greift der Endpoint direkt auf PyLucene und den lokalen Index zu:

```
from flask import Flask, request, Response

from java.nio.file import Paths
from org.apache.lucene.store import FSDirectory
from org.apache.lucene.index import DirectoryReader, Term
from org.apache.lucene.analysis.standard import StandardAnalyzer
from org.apache.lucene.search import (
    IndexSearcher,
    BooleanQuery,
    BooleanClause,
    TermQuery,
    MatchAllDocsQuery,
)
```

Die zentrale Route ist `/sru`, ergänzt um einen einfachen Gesundheitscheck:

```
@app.route("/health", methods=["GET"])
def health():
    try:
        _ensure_lucene()
        app.searcher.search(MatchAllDocsQuery(), 1)
        return {"status": "ok"}, 200
    except Exception as e:
        return {"status": "error", "detail": str(e)}, 500
```

Diese Route diene vor allem dazu, schnell zu prüfen, ob die Flaskanwendung läuft, die Lucene-VM initialisiert ist, und der Index geöffnet werden kann.

Für den eigentlichen Endpoint wurden zwei SRU-nahe Routen implementiert:

```
@app.route("/sru", methods=["HEAD"])
def sru_head():
    return set_sru_headers(Response(status=200), SRU_VERSION)

@app.route("/sru", methods=["GET"])
def sru_search():
    _ensure_lucene()
    sru_ver = requested_sru_version()
```

- `HEAD /sru` wird benutzt, um Header und Erreichbarkeit ohne Body zu prüfen.
- `GET /sru` übernimmt die eigentliche Anfrageverarbeitung.

Im GET-Handler werden dann mehrere Fälle unterschieden:

- SRU-Version prüfen
- FCS spezifischen Probeaufruf behandeln
- operation=explain beantworten
- operation=searchRetrieve ausführen
- nicht unterstützte Operationen als Diagnostic zurückgeben

Die Logik im Kern sieht so aus:

```
if request.args.get("x-fcs-endpoint-description") is not None:
    xml = fcs_explain_xml(app.explain_meta, version=SRU_VERSION)
    return xml_response(xml, SRU_VERSION, 200)

if op == "explain":
    xml = fcs_explain_xml(app.explain_meta, version=SRU_VERSION)
    return xml_response(xml, SRU_VERSION, 200)

if op not in {"searchretrieve", ""}:
    diag = sru_diagnostic_xml(
        code="7",
        message="Unsupported operation",
        details=f"operation={op_raw}",
        version=SRU_VERSION,
    )
    return xml_response(diag, SRU_VERSION, 200)
```

Hier ist bereits gut sichtbar, wofür der erste Endpoint-Versuch gedacht war:

- normale Explain Aufrufe beantworten
- auch den FCS-typischen Probeaufruf x-fcs-endpoint-description=true bedienen

ansonsten die Suchlogik ausführen.

9.3 XML-Erzeugung als eigener Baustein

Ein zentraler Bestandteil dieses Versuchs war, dass die XML Ausgabe nicht von einer fertigen Endpoint-Bibliothek übernommen wurde, sondern in `scripts/FCS/fcs_xml.py` selbst erzeugt wurde.

Dort wurden drei zentrale Funktionen benutzt:

- `fcs_explain_xml(...)`
- `fcs_searchretrieve_xml(...)`
- `sru_diagnostic_xml(...)`

Während `fcs_endpoint.py` den Ablauf und die Anfrageverarbeitung steuert, kapselt `fcs_xml.py` die formale XML-Ausgabe. Diese Teilung ermöglichte, verschiedene Aspekte des Endpoints nach Schwerpunkt zu steuern.

Für `searchRetrieve` bedeutete das: Die vom Searcher gefundenen Treffer mussten zunächst in interne Python-Strukturen konvertiert und anschließend in XML serialisiert werden. Ein Treffer wurde dabei wie gewöhnt als Kombination aus Titel, Identifier/URL, Magazin, Ausgabenlabel, Datum und der KWIC-Darstellung weitergereicht.

Im Endpoint wurde dafür pro Treffer ein Dictionary aufgebaut:

```
records.append({
    "id": url or str(sd.doc),
    "title": title,
    "magazine": mag,
    "issue_label": label,
    "issue_date_iso": dateiso,
    "kwic": kwic_list,
})
```

Diese Daten wurden anschließend an `fcs_searchretrieve_xml(...)` übergeben, das daraus eine SRU-Response mit `fcs:Resource`, `fcs:ResourceHeader`, `fcs:extent` und `fcs:DataView` erzeugte.

9.4 Explain-Metadaten und FCS-Beschreibung

Ein weiterer wichtiger Bestandteil war die interne Beschreibung des Endpoints.

Diese wurde in `fcs_endpoint.py` als Metadatenstruktur am Flask-Objekt hinterlegt:

```
app.explain_meta = {
    "server_info": {
        "base_url": os.environ.get("FCS_BASEURL", "https://starts-human-
mph-beyond.trycloudflare.com/sru"),
        "title": "DigitProject FCS Endpoint",
        "description": "FCS 2.0 Test Endpoint (local)",
        "contact": "mailto:you@example.com",
        "version": "2.0",
    },
}
```

```

"supports": {
  "queryLanguages": ["cqlfcs-2.0", "cql"],
  "dataViews": ["hits:kwic-1.0", "fcs:resource"]
},
...
}

```

Diese Struktur wurde im Explain-Response ausgegeben und sollte beschreiben,

- welche Art von Endpoint vorliegt
- welche Data Views unterstützt werden
- welche Felder vorhanden sind
- welche Sprache bzw. Standards erwartet werden

Der Explain-Aufruf lieferte lokal etwa folgende Struktur im XML:

```

<fcs:EndpointDescription>
  <fcs:Capabilities>
    <fcs:capability>searchRetrieve</fcs:capability>
    <fcs:capability>explain</fcs:capability>
  </fcs:Capabilities>
  <fcs:SupportedDataViews>
    <fcs:dataView>hits:kwic-1.0</fcs:dataView>
  </fcs:SupportedDataViews>
  <fcs:Collections>
    <fcs:Collection>
      <fcs:id>zx-corpus</fcs:id>
      <fcs:label>ZX Press Corpus</fcs:label>
    </fcs:Collection>
  </fcs:Collections>
  <fcs:Languages>
    <fcs:language>ru</fcs:language>
    <fcs:language>en</fcs:language>
    <fcs:language>de</fcs:language>
  </fcs:Languages>
</fcs:EndpointDescription>

```

9.5 Verbindung zwischen CQL und Lucene

Ein praktisches Kernproblem des Endpoints bestand darin, externe Suchanfragen in eine Form zu überführen, die intern gegen Lucene ausführbar ist.

Dafür wurde `scripts/FCS/cql_parser.py` als eine CQL-Zwischenschicht vorgesehen.

Im Endpoint wurde dies über eine Funktion wie `cql_to_lucene(...)` angebunden:

```
raw_query = (request.args.get("query") or "").strip()
query_echo = raw_query

qtext = cql_to_lucene(raw_query) if raw_query else ""
```

Danach wurde aus dem Suchtext eine Lucene-Abfrage erzeugt. Für die eigentliche Querybildung wurde gleiche wie im Suchprototyp (*vgl. Kap. 8*) entwickelte Funktion `build_query(...)` wiederverwendet.

Der Kern der Query sieht so aus:

```
qry = build_query(
    qtext=qtext,
    magazine=magazine,
    form=form_,
    lang=language,
    year_from=y_from,
    year_to=y_to,
)
```

9.6 Trefferdarstellung und KWIC im Endpoint

Der Endpoint sollte gleich wie im Suchprototyp die Suchtreffer nach Möglichkeit nicht nur als Liste von Titeln, sondern mit einer KWIC Darstellung ausgeben.

Dafür wurde direkt auf die bereits vorhandenen Hilfsfunktionen aus `scripts/TextSearch/Searcher.py` zurückgegriffen.

Im Endpoint wurde zunächst ein KWIC-Begriff bestimmt:

```
raw_kw = (kwic_term or qtext or "").strip()
term_for_kwic = _normalize_kwic_term(raw_kw)
```

Danach wurde versucht, für den jeweiligen Dokumenttext kurze Kontextausschnitte zu gewinnen:

```
snips = kwic(content, term_for_kwic, window=kwic_window, max_snips=3) or  
[]  
if not snips:  
    fb = _fallback_snippet(content, term_for_kwic, chars=180)  
    if fb:  
        snips = [fb]
```

Es wurden:

- echte wortnahe Kontextausschnitte bevorzugt,
- notfalls Fallback-Snippets gegeben, wenn die Funktion nichts passendes findet.

Die Snippets wurden danach in linke, mittlere und rechte Kontexte zerlegt und als `hits:kwic-1.0 DataView` ausgegeben.

Nach der Implementierung des Endpoints wurde dieser schrittweise auf drei Ebenen geprüft: zuerst lokal auf Erreichbarkeit und XML-Antworten, anschließend funktional auf Explain und SearchRetrieve Verhalten und schließlich öffentlich über einen Tunnel, um externe Zugriffe und den Validator testen zu können.

9.7 Befehle zum Starten und Testen

Der Endpoint wurde damit gestartet:

```
cd ../../Pfad zum Projektordner (in Pycharm)  
conda activate Dig-Project-arm64 #manuelle Umgebung angepasst an die Java-  
Anforderungen auf MacOS  
python scripts/FCS/fcs_endpoint.py
```

Zur Prüfung wurden anschließend lokale curl-Aufrufe eingesetzt. Typische Testbefehle:

```
curl -i "http://127.0.0.1:8088/health"  
curl -I "http://127.0.0.1:8088/sru"  
curl -i "http://127.0.0.1:8088/sru?operation=explain&version=2.0"  
curl -i "http://127.0.0.1:8088/sru?x-fcs-endpoint-description=true"
```

Die Testbefehle decken unterschiedliche Funktionsebenen ab.

`/health` diene als einfacher Laufzeittest für Flask und Lucene.

`HEAD /sru` wurde verwendet, um die SRU-bezogenen HTTP-Header ohne vollständige XML-Antwort zu prüfen.

`operation=explain` sollte die Fähigkeiten und Metadaten des Endpoints zurückgeben

`operation=searchRetrieve` führte die eigentliche Suche gegen den Index aus.

Output-Beispiele:

```
# HEAD /sru
HTTP/1.1 200 OK
Content-Type: application/sru+xml;version=2.0; charset=utf-8
X-SRU-Version: 2.0
#...
#operation=explain
<sru:explainResponse ...>
  <sru:version>2.0</sru:version>
```

Für die eigentliche Suche wurden etwa folgende Befehle verwendet:

```
curl -i
"http://127.0.0.1:8088/sru?operation=searchRetrieve&version=2.0&query=cql.
serverChoice=%22%D1%81%D0%BF%D0%B5%D0%BA%D1%82%D1%80%D1%83%D0%BC%22&maximu
mRecords=3"
```

sowie feldspezifische Anfragen wie:

```
curl -i
"http://127.0.0.1:8088/sru?operation=searchRetrieve&version=2.0&query=titl
e=%22%D0%A1%D0%BF%D0%B5%D0%BA%D1%82%D1%80%D1%83%D0%BC%22&maximumRecords=3"
```

„`%D1%81%D0%BF%D0%B5%D0%BA%D1%82%D1%80%D1%83%D0%BC`“ ist dabei die URL-Encoding für das russische Wort **спектрум**, um kyrillische Suchausdrücke und CQL-Parameter sicher über HTTP zu übertragen.

9.8 Öffentliche Erreichbarkeit über Tunnel

Da ein externer Validator nicht auf `localhost` zugreifen kann, musste der Endpoint zeitweise öffentlich erreichbar gemacht werden.

Dafür wurden Tunnel-Lösungen verwendet, zunächst `Cloudflare Quick-Tunnels`, später stabiler `ngrok`.

Der `ngrok`-Start lautete:

```
ngrok http http://127.0.0.1:8088
```

Die erzeugte öffentliche Basis-URL war in einem Testlauf:

```
Forwarding https://unrevolted-leonor-willfully.ngrok-free.dev ->
http://127.0.0.1:8088
```

Darüber wurden dieselben Requests erneut ausgeführt, etwa:

```
curl -I "https://unrevolted-leonor-willfully.ngrok-free.dev/sru"
curl -i "https://unrevolted-leonor-willfully.ngrok-free.dev/sru?operation=explain&version=2.0"
curl -i "https://unrevolted-leonor-willfully.ngrok-free.dev/sru?x-fcs-endpoint-description=true"
curl -i "https://unrevolted-leonor-willfully.ngrok-free.dev/sru?operation=searchRetrieve&version=2.0&query=cql.serverChoice=%22D1%81%D0%BF%D0%B5%D0%BA%D1%82%D1%80%D1%83%D0%BC%22&maximumRecords=3"
```

Dabei antwortete Endpoint lokal und über `ngrok` konsistent mit den lokalen Tests:

- Content-Type: application/sru+xml;version=2.0; charset=utf-8
- X-SRU-Version: 2.0

Ein Explain-Request wie

```
curl -i „http://127.0.0.1:8088/sru?operation=explain&version=2.0“
```

lieferte eine `explainResponse` mit `SRU-Version 2.0`, `FCS EndpointDescription`, unterstützten `Data Views`, Sprachen, Feldern und Metadaten.

Ein Suchrequest mit `cql.serverChoice="спектрум"` (de: spectrum) lieferte hingegen eine `searchRetrieveResponse` mit echten Treffern aus dem Korpus, z. B. Artikeln wie:

- Body 32 - scene more - где взять **спектрум** в минске. (de: wo kann man sich in Minsk ein **Spectrum** holen.)
- World in the Web 01 - **Спектрум** сегодня - **спектрум** не умер, а процветает... (de: **Spectrum** heute - **spectrum** hat nicht gestorben, sondern blüht auf...)

Vollständiger Output:

```
<sru:numberOfRecords>1715</sru:numberOfRecords>
<fcs:ResourceHeader>
  <fcs:title>World in the Web 01 - Спектрум сегодня - спектрум не
  умер...</fcs:title>
  <fcs:identifier>https://zpress.ru/article.php?id=12792</fcs:identifier>
</fcs:ResourceHeader>
```

Und für KWIC:

```
<hits:kwic>
  <hits:leftContext>World in the Web 01 - </hits:leftContext>
  <hits:match>Спектрум</hits:match>
  <hits:rightContext> сегодня - спектрум не умер...</hits:rightContext>
</hits:kwic>
```

Damit wurde klar, dass die Volltextsuche gegen den realen Korpus funktionierte und Treffer öffentlich ausgab.

9.9 Fachliche Grenzen des ersten Endpoints

Trotz dieses funktionierenden Zustands war der erste Versuch noch nicht fachlich vollständig ausgereift.

Erstens war die feldspezifische Suche noch nicht in allen Fällen präzise. Anfragen wie `title="Спектрум"` oder andere titelorientierte Suchen wurden zwar verarbeitet, lieferten aber nicht immer Treffer, deren Titel semantisch überzeugend zum Suchausdruck passten. Das deutet darauf hin, dass die CQL-zu-Lucene Übersetzung bzw. die Feldbindung im ersten Versuch noch ungefähr und nicht vollständig streng umgesetzt war.

Zweitens war die KWIC-Ausgabe zwar XML-seitig vorhanden, aber inhaltlich nicht immer ideal. In manchen Responses erschien im match-Segment der gesamte Titel oder das Fallback statt eines sauber aus dem Volltext ausgeschnittenen linken und rechten Kontextes.

Die Data View war also technisch vorhanden, aber eine konkrete Kontextdarstellung blieb teilweise prototypisch.

Drittens verlagerte sich die Problemzone des Projekts in dieser Phase zunehmend von der Suchfunktion auf die Protokollkonformität.

Die eigentliche Volltextsuche lief, ebenso `Explain` und `SearchRetrieve`. Die Schwierigkeiten entstanden vor allem bei:

- Versionsbehandlung
- Headern
- Explain-Autodetect
- x-fcs-endpoint-description
- Erwartungen des externen Validators

Zur Prüfung der Konformität wurde am Ende neben lokalen curl-Tests der FCS Endpoint Validator eingesetzt, konkret der Leipziger Validator unter: <https://fcs-validator.data.saw-leipzig.de> (es wurde auf der Clarin-Webseite verlinkt).

Dabei handelt es sich um einen webbasierten FCS/SRU-Validator der Sächsischen Akademie der Wissenschaften zu Leipzig. Das Werkzeug prüft automatisiert, ob ein Endpoint in Aufbau, Versionierung und Antwortformat den Erwartungen des FCS-Protokolls entspricht.

Die Idee war, die öffentliche Tunnel-URL als BaseURL einzutragen und den Endpoint automatisiert auf die Konformität prüfen zu lassen.

Es wurde beim konkreten Lauf gegen Validator nicht erfolgreich vollständig validiert:

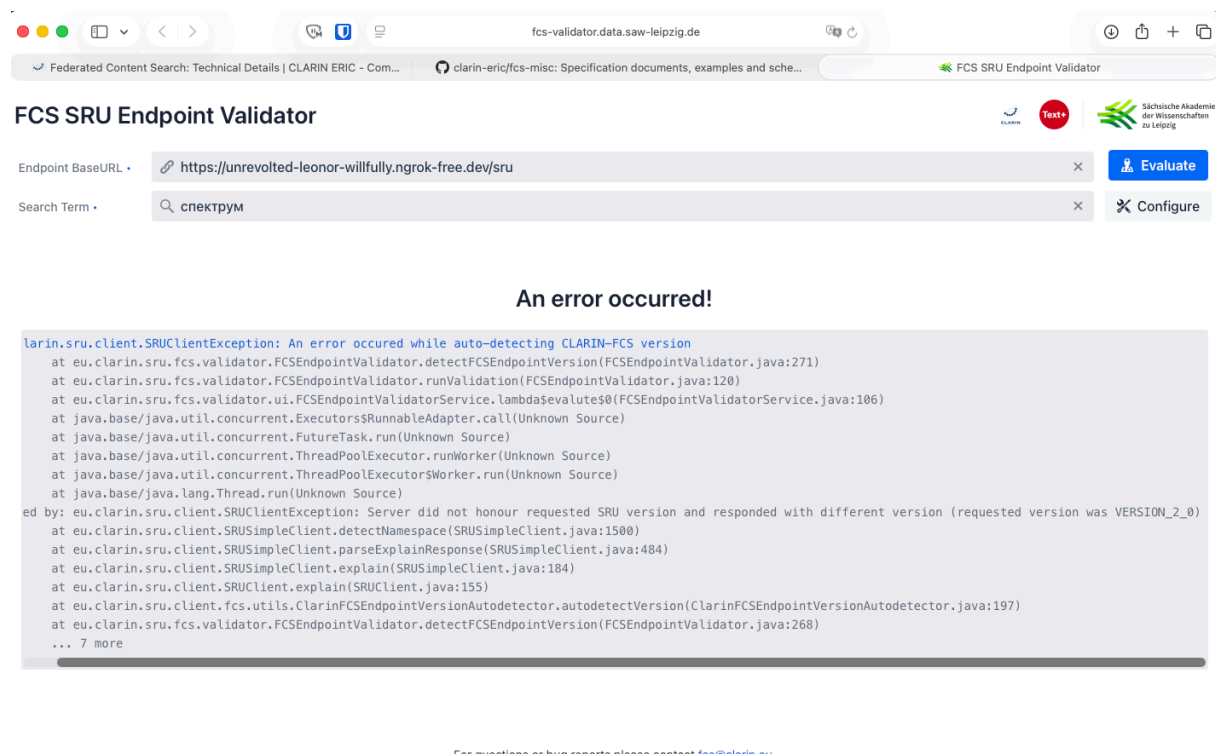


Abb.2 - Fehlermeldung auf der Validatorseite

Die genaue technische Ursache dafür wurde dabei nicht abschließend erkannt.

9.10 Ergebnis des ersten FCS-Versuchs

Am Ende dieses ersten Versuchs lag also unter `scripts/FCS/` ein eigener Flask-basierter SRU-/FCS-Prototyp vor, der:

- den Lucene-Index direkt ansprach
- Explain und SearchRetrieve implementierte
- lokal und öffentlich erreichbar war
- FCS-nahe XML-Antworten ausgab
- KWIC und Fallback-Strukturen nutzte

aber nicht erfolgreich bis zur endgültigen externen Validator-Konformität abgeschlossen werden konnte.

Die Resultaten brachten damit viele Unklarheiten. Die Suche selbst war dabei bereits nutzbar, die offene Problemzone lag zuletzt vor allem in der exakten Protokoll- und Validatoranpassung.

Diese Sackgasse bildete den Ausgangspunkt für den späteren zweiten Hauptversuch unter `scripts/FCS_Server/`. Anders als im ersten Ansatz sollte die Protokollschicht nun nicht mehr weitgehend selbst gebaut, sondern stärker an einen bibliotheksgestützten CLARIN-/SRU-Serverrahmen angelehnt werden (*vgl. Kap. 10*).

10. Zweiter Versuch eines FCS-Endpoints (scripts/FCS_Server/)

Die bereits vorhandene Suchlogik sollte nicht mehr vollständig in einer selbstgebauten Flask-Protokollschicht betrieben werden, sondern in einen bibliotheksgestützten SRU und FCS-Serverrahmen überführt werden.

Dafür wurde versucht, die äußere Server- und Protokollschicht stärker an einer bestehenden Referenzarchitektur auszurichten.

10.1 Orientierung an CLARIN-Referenzprojekten

Für diesen zweiten Ansatz wurde sich nicht mehr nur an der allgemeinen FCS-Spezifikation orientiert, sondern an einer konkreten Referenzimplementierung für FCS 2.0 in Python, nämlich dem Projekt `fcs-korp-endpoint-python` (Empfohlen von der Clarin-Spezifikationsseite).

Wichtige Referenz war dabei insbesondere das Repository <https://github.com/Querela/fcs-korp-endpoint-python>

Die README dieses Projekts beschreibt die Methode ausdrücklich als „FCS 2.0 reference endpoint implementation“ und als Prototyp für neue Endpoints.

Dabei wird dort erklärt, dass:

`app.py` die SRU-/FCS-Konfiguration setzt,

`__main__.py` den lokalen Start über `python3 -m ...` zeigt,

`sru-server-config.xml` und `endpoint-description.xml` für den jeweiligen eigenen Endpoint angepasst werden müssen,

`endpoint.py` die eigentliche Endpoint-Implementierung mit `explain` und `searchRetrieve` enthält.

Zentrale Dateien dieses Strangs sind:

- `scripts/FCS_Server/__main__.py`
- `scripts/FCS_Server/app.py`
- `scripts/FCS_Server/zx_search_engine.py`
- `scripts/FCS_Server/sru-server-config.xml`
- `scripts/FCS_Server/endpoint-description.xml`

Der zweite Versuch ist deshalb am besten als Hybrid zu beschreiben: übernahme einer bibliotheksgestützten SRU-/FCS-Serverhülle, aber die Weiterverwendung der Lucene- und KWIC-Logik (vgl. Kap. 8-9).

10.2 Grundidee der Hybridarchitektur

Die Grundidee dieses zweiten Ansatzes bestand darin, die äußere Serverstruktur aus einer Referenzarchitektur zu übernehmen, den eigentlichen Suchkern aber weiterhin projektintern zu belassen. Genau deshalb griff `zx_search_engine.py` weiterhin direkt auf den lokalen Lucene-Index und auf bereits vorhandene Projektfunktionen zurück.

Dazu gehörten insbesondere:

- `build_query(...)`
- `_normalize_kwic_term(...)`
- `kwic(...)`
- `_fallback_snippet(...)`

Diese Bausteine waren bereits im ersten Such- und Endpoint-Kontext entwickelt worden und wurden im zweiten Versuch wiederverwendet (vgl. Kap. 8.5.-8.7; 9.5-9.6).

Am Anfang von `zx_search_engine.py` zeigt sich diese direkte Kopplung vor:

```
from scripts.FCS.fcs_endpoint import build_query, _normalize_kwic_term
from scripts.TextSearch.Searcher import kwic, _fallback_snippet

BASE_DIR = Path(__file__).resolve().parents[2]
INDEX_DIR = str(BASE_DIR / "index_dir")
```

Der Startpunkt wurde über `__main__.py` und `app.py` organisiert.

Der modulare Start erfolgt in `__main__.py`:

```
from werkzeug.serving import run_simple
from .app import make_app

if __name__ == "__main__":
    app = make_app()
    run_simple("127.0.0.1", 8088, app, use_debugger=True,
              use_reloader=False)
```

app.py erzeugt den bibliotheksgestützten Serverrahmen über SRUServerErrorApp(...):

```
from clarin.sru.server.wsgi import SRUServerErrorApp
from clarin.sru.server.config import SRUServerErrorConfigKey

from scripts.FCS_Server.zx_search_engine import ZXSearchEngine
```

Dabei wurde dort eine make_app()-Funktion benutzt, die unter anderem diese Parameter setzte:

```
SRUServerErrorConfigKey.SRU_TRANSPORT: "http",
SRUServerErrorConfigKey.SRU_HOST: "127.0.0.1",
SRUServerErrorConfigKey.SRU_PORT: "8088",
SRUServerErrorConfigKey.SRU_DATABASE: "zxpress",
SRUServerErrorConfigKey.SRU_ECHO_REQUESTS: "true",
SRUServerErrorConfigKey.SRU_NUMBER_OF_RECORDS: 250,
SRUServerErrorConfigKey.SRU_MAXIMUM_RECORDS: 1000,
SRUServerErrorConfigKey.SRU_ALLOW_OVERRIDE_MAXIMUM_RECORDS: "true",
SRUServerErrorConfigKey.SRU_ALLOW_OVERRIDE_INDENT_RESPONSE: "true",
SRUServerErrorConfigKey.SRU_SUPPORTED_VERSION_MIN: "2.0",
SRUServerErrorConfigKey.SRU_SUPPORTED_VERSION_MAX: "2.0",
SRUServerErrorConfigKey.SRU_SUPPORTED_VERSION_DEFAULT: "2.0",
```

An diesem Punkt war der erhaltene Projektzustand zunächst inkonsistent.

Frühere Zustände wiesen folgende Probleme auf:

- unpassende absolute Altpfade
- eine nicht saubere Kopplung zwischen __main__.py und app.py
- unvollständige Serverparameter

Im Verlauf des Projektes wurden diese Punkte schrittweise bereinigt, bis der Server überhaupt wieder startbar war.

10.3 Endpoint-Beschreibung und XML-Dateien

Zur äußeren Server- und Beschreibungsstruktur gehörten, gleich wie in der Referenzimplementierung, zwei zentrale XML-Dateien:

- scripts/FCS_Server/sru-server-config.xml
- scripts/FCS_Server/endpoint-description.xml

endpoint-description.xml wurde im Verlauf der Endpointeinstellung besonders wichtig. Eine frühere Fassung war nicht mit dem verwendeten Parser kompatibel, weil sie in einem veralteten bzw. unpassenden Namespace-Format stand. Erst nach einer Umstellung auf eine Core-2-artige Struktur wurde sie vom bibliotheksgestützten Server akzeptiert.

Der Beschreibung des Endpoints orientierte sich schließlich an diesem Modell:

```
<EndpointDescription
  xmlns="http://clarin.eu/fcs/endpoint-description"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  version="2">
```

Darin wurden unter anderem definiert:

- Capabilities,
- SupportedDataViews,
- SupportedLayers,
- Resource-Beschreibung für das ZX-Press-Korpus.

Beispiele der Beschreibungen:

```
<Capabilities>
  <Capability>http://clarin.eu/fcs/capability/basic-search</Capability>
</Capabilities>

<Resource pid="zx-corpus">
  <Title xml:lang="en">ZX Press Corpus</Title>
  <Description xml:lang="en">
    Digital corpus of Russian ZX Spectrum disk magazines and related
    articles.
  </Description>
```

Die Datei `sru-server-config.xml` wurde ebenfalls mehrmals bereinigt und projektbezogen angepasst. Frühere Fremdrete aus dem Referenzkontext wurden durch projektbezogene Metadaten ersetzt.

Die zentralen Einträge darin sind:

```
<title xml:lang="de" primary="true">Diskmags Korpus ZX-Press</title>
<title xml:lang="en">ZX Press Corpus</title>
<name set="fcs">text</name>
```

10.4 Rolle von `zx_search_engine.py`

Während `app.py`, `__main__.py` und die XML-Dateien die äußere Serverhülle bereitstellen sollten, lag der eigentliche Verbindungspunkt zwischen dieser Hülle und dem lokalen Suchkern in der Adapterdatei `zx_search_engine.py`.

Die Klasse `ZXSearchEngine` sollte als Suchmaschine innerhalb von `SRUServerApp` fungieren.

In dieser Datei waren insbesondere folgende Aufgaben verbunden:

- das Laden der `endpoint-description.xml`
- die Initialisierung der Lucene-VM
- der Zugriff auf `DirectoryReader` und `IndexSearcher`
- die Übernahme von Suchparametern daraus
- die Generierung von KWIC-Snippets
- die Rückgabe von Treffern in einer Python-Struktur

Der Kern der Querybildung sieht dort so aus:

```
lucene_q = (build_query(
    qtext=raw_query, magazine=mag, form=form_, lang=lang,
    year_from=int(y_from) if y_from else None,
    year_to=int(y_to) if y_to else None
) if raw_query else MatchAllDocsQuery())
```

Auch die KWIC-Erzeugung wurde direkt aus dem ersten Suchstrang übernommen:

```
snips = kwic(content, kwic_term, window=kwic_window, max_snips=3) or []
if not snips:
    fb = _fallback_snippet(content, kwic_term, chars=180)
    if fb:
        snips = [fb]
```

Pro Treffer wurde dann eine Datenstruktur dieser Form aufgebaut:

```
records.append({
    "identifier": identifier,
    "title": title,
    "extents": [
        {"type": "magazine", "value": doc.get("magazine") or ""},
        {"type": "issue", "value": doc.get("issue_label") or ""},
        {"type": "date", "value": doc.get("issue_date_iso") or ""},
    ],
    "dataViews": [
        {"type": "hits:kwic-1.0", "payload": {"kwic": snippets}}
    ]
})
```

10.5 Vergleich mit der Querela(Clarin)-Referenz

Gerade an diesem Punkt wurde der Vergleich mit der Referenz besonders aufschlussreich. Dabei zeigte sich, dass der zweite Versuch zwar strukturell an die Referenz angenähert war, übernahm aber ihr internes Ausführungsmodell aber nicht vollständig.

Die größten Unterschiede lassen sich so zusammenfassen:

`main.py` und `app.py` sind der Referenz strukturell relativ nahe.

Die eigentliche Referenzimplementierung besitzt mit `endpoint.py` einen klar ausgearbeiteten Endpoint-Kern. Dort werden Requests nicht nur angenommen, sondern über ein eigenes Request-, Query- und ResultSet-Modell vollständig weiterverarbeitet.

`scripts/FCS_Server/` besitzt kein wirkliches Gegenstück zu diesem vollständigen `endpoint.py`. Stattdessen sollte `zx_search_engine.py` gleichzeitig Adapter, Suchmaschine und Ergebnislieferant sein.

Die Referenz liefert echte `SRUSearchResultSet` Objekte zurück. Im eigenen Endpoint wurden dagegen vereinfachte Python-Dictionaries zurückgegeben.

Die Referenz arbeitet auf einem expliziten `SRURequestModell` mit Query-Typebehandlung. Der eigene Adapter behandelte Requests dagegen deutlich vereinfachter.

Zusammengefasst hat der zweite Versuch hat die Serverhülle der Referenz weit übernommen, aber den inneren `ResultSet`-Pfad nicht vollständig mitportiert.

10.6 Die Reparaturschritte

Im praktischen Test zeigte sich, dass dieser Endpoint nicht unmittelbar lauffähig war. Bevor überhaupt Suchanfragen geprüft werden konnten, mussten daher mehrere Minimalanpassungen vorgenommen werden.

Im Verlauf der Arbeit wurden nach dem Testen mehrere konkrete Minimalreparaturen vorgenommen, um den zweiten Strang wieder testbar zu machen.

Dazu gehörten insbesondere:

- Vereinheitlichung von `__main__.py` und `app.py` über `make_app()`
- Ersetzung harter Altpfade
- Ergänzung fehlender SRU-Parameter
- Implementierung bzw. Ergänzung von `create_EndpointDescription(...)`
- Ersetzung einer inkompatiblen `endpoint-description.xml`
- schrittweise Anpassung von `sru-server-config.xml`

10.7 Testbefehle

Zur Prüfung wurden anschließend lokale curl-Aufrufe eingesetzt. Verwendet wurden unter anderem:

```
curl -i "http://127.0.0.1:8088/?operation=explain&version=2.0"
curl -i
"http://127.0.0.1:8088/?operation=searchRetrieve&version=2.0&query=cql.serverChoice=%22%D1%81%D0%BF%D0%B5%D0%BA%D1%82%D1%80%D1%83%D0%BC%22&maximumRecords=3"
curl -i
"http://127.0.0.1:8088/?operation=searchRetrieve&version=1.2&query=cql.serverChoice=%22%D1%81%D0%BF%D0%B5%D0%BA%D1%82%D1%80%D1%83%D0%BC%22&maximumRecords=3"
```

Zusätzlich wurden Explain-Tests mit und ohne Versionsparameter sowie mehrere kleine Varianten des SearchRetrieve-Aufrufs ausprobiert.

10.8 Beobachtete Outputs

Die entscheidenden Befunde ergaben sich erst aus den lokalen Testaufrufen. Sie zeigten, dass der Server zwar grundsätzlich startete, die Suchanfragen jedoch nicht in eine funktionierende `searchRetrieve`-Ausführung überführt wurden.

Ein typischer Output enthielt:

```
<sruResponse:explainResponse ...>  
<sruResponse:version>2.0</sruResponse:version>
```

Gleichzeitig wurde jedoch eine Diagnostik mit ausgegeben:

```
<diag:diagnostic>  
  <diag:uri>info:srw/diagnostic/1/5</diag:uri>  
  <diag:details>1.2</diag:details>  
  <diag:message>Version '2.0' is not supported</diag:message>  
</diag:diagnostic>
```

Bei einem Aufruf mit `version=1.2` erschien spiegelbildlich:

```
<diag:details>2.0</diag:details>  
<diag:message>Version '1.2' is not supported by this  
endpoint.</diag:message>
```

Noch wichtiger war der Suchtest.

```
curl -i  
"http://127.0.0.1:8088/?operation=searchRetrieve&version=2.0&query=cql.serverChoice=%22%D1%81%D0%BF%D0%B5%D0%BA%D1%82%D1%80%D1%83%D0%BC%22&maximumRecords=3"
```

Der Request lieferte nicht die erwartete `searchRetrieveResponse`, sondern erneut eine `explainResponse` mit Versionsdiagnostik.

Damit ergab sich ein sehr klarer technischer Befund: der Server startete, der Explain-Zweig wurde erreicht, aber `searchRetrieve` gelangte nicht sauber in eine funktionierende Suchausführung.

Dieser zweite Versuch scheiterte damit an einer anderen Stelle als der erste.

Beim ersten Endpoint auf Flask war die eigene Suchlogik funktional bereits benutzbar. Die Schwierigkeiten lagen vor allem in der Protokollkonformität gegenüber externen Quellen.

Beim zweiten Versuch verschob sich das Problem tiefer in die Integrationsschicht. Hier war die Suchlogik zwar weiterhin vorhanden, wurde aber nicht mehr zuverlässig durch die bibliotheksgestützte Protokollschicht erreicht.

Aus dem Vergleich mit der Referenz und den ausgeführten Tests lässt sich ableiten, dass das Hauptproblem nicht in den Korpusdaten, nicht im Index und auch nicht primär in der Lucene-Logik lag.

Die Ursache ist daher eher in der FCS-Versionsaushandlung, der Request-Weiterleitung innerhalb des Bibliotheksrahmens, der unvollständigen Übernahme des internen ResultSet-Modells oder allgemein in der Hybridisierung von Referenzhülle und eigenem Suchadapter zu vermuten.

Am Ende dieses zweiten Versuchs lag unter `scripts/FCS_Server/` also kein fertiger Ersatz für den ersten Endpoint vor, sondern ein fortgeschrittener, aber nicht abgeschlossener Umbauversuch. Damit dokumentiert der zweite Endpoint keine produktiv abgeschlossene Lösung, sondern eher einen fortgeschrittenen Migrationsversuch zwischen Eigenimplementierung und Referenzarchitektur. Für das Projekt ist gerade dieser Befund wichtig: Er zeigt, dass die offene Problemzone nicht mehr in der lokalen Suchfunktion lag, sondern in der Einbindung dieser Suchfunktion in einen standardisierten bibliotheksgestützten Protokollrahmen.

Quellen- und Ressourcenverzeichnis

Externe Online-Ressourcen und Spezifikationen

- ZXpress
- CLARIN Technical Details
- FCS Core Specification 2.0
- FCS Data View Specification 1.0
- fcs-misc
- FCS Endpoint Validator
- fcs-korp-endpoint-python

Projektinterne Dateien

- scripts/light/run_light_pipeline.py
- scripts/light/scrape_issue_listing_light.py
- scripts/light/fetch_articles_light.py
- scripts/light/patch_metadata_light.py
- validate_corpus.py
- audit_corpus.py
- scripts/TextSearch/Indexer.py
- scripts/TextSearch/Searcher.py
- scripts/TextSearch/_tools/full_healthcheck.py
- scripts/FCS/fcs_endpoint.py
- scripts/FCS/fcs_xml.py
- scripts/FCS/cql_parser.py
- scripts/FCS_Server/app.py
- scripts/FCS_Server/zx_search_engine.py
- scripts/FCS_Server/sru-server-config.xml
- scripts/FCS_Server/endpoint-description.xml

Projektinterne Logs und Datenbeispiele

- logs/audit_summary.csv
- logs/health/summary.txt
- logs/health/magazines_empty.log
- logs/validation/validate_47th_Byte_20250923_212149.txt

- Beispielpfade unter data/zxpress/...